用户指南
CLB 工具

摘要

本用户指南介绍可配置逻辑块（CLB）工具的结构及使用方法。假定读者已熟悉 CLB 架构和 CCS IDE。有关 CLB 架构的信息，请参阅器件专用技术参考手册（TRM）。

目录

1 引言 3
2 入门 5
3 使用 CLB 工具 10
4 示例 25
5 在现有 DriverLib 项目中启用 CLB 工具 41
6 常见问题解答（FAQs） 45
7 修订历史 45

插图目录

图 1-1：将 CLB 逻辑单元设计连接到 CLB 实例 4
图 1-2：CLB 工具项目结构 4
图 1-3：CLB 工具构建流程 5
图 2-1：TDM 编译器安装向导 6
图 2-2：更改 TDM 许可 7
图 2-3：TDM 编译器路径 7
图 2-4：TDM 组件 8
图 2-5：创建 SystemC 目录 8
图 2-6：SystemC 配置输出 9
图 2-7：Make 输出 9
图 2-8：Make install 输出 10
图 3-1：导入 CCS Eclipse 项目 10
图 3-2：链接资源 11
图 3-3：构建变量 11
图 3-4：用于生成图表的构建变量 12
图 3-5：CLB 工具 SysConfig 界面 12
图 3-6：边界输入选项 13
图 3-7：用户描述文本框 13
图 3-8：计数器选项 14
图 3-9：方程警告 14
图 3-10：CLB 工具生成的文件 15
图 3-11：“clb.h”头文件示例 15
图 3-12：CLB 方框图 16
图 3-13：HLC 配置示例 16
图 3-14：GENERATE_DIAGRAM 构建变量 17
图 3-15：静态选项 17
图 3-16：边界输入 0 至 7 18
图 3-17：边界输入方波 18
图 3-18：自定义边界输入 19
图 3-19：边界输入连接至 tile 输出 20
图 3-20：CLB 仿真示例 20
图 4-1：示例 3：生成的 PWM 波形 25
图 4-2：示例 1：EPWM 同步 27
图 4-3：示例 1：PWM 测试模式 28
图 4-4：示例 1：逻辑图 28

Copyright © 2023 Texas Instruments Incorporated

Figure 4-5. 示例 1：生成的 PWM.....29 Figure 4-6. 示例 1：CLB 配置..... 30 Figure 4-7. 示例 2：GPIO 毛刺示例..... 31 Figure 4-8. 示例 2：CLB 配置..... 31 Figure 4-9. 示例 2：GPIO 毛刺宽度..... 32 Figure 4-10. 示例 5：事件窗口配置..... 34 Figure 4-11. 示例 6：占空比超过预设值..... 36 Figure 4-12. 示例 6：周期超过预设值..... 36 Figure 4-13. 示例 17：单次触发 PWM 输出..... 38 Figure 4-14. 示例 17：CLB 总体配置..... 38 Figure 5-1. 启用 SysConfig.....41 Figure 5-2. 构建后步骤.....41 Figure 5-3. 启用 CLB 工具所需的链接资源.....42 Figure 5-4. SysConfig SDK 路径..... 42 Figure 5-5. 支持 CLB 工具的 epwm_ex1_trip_zone.....44 Figure 6-1. 仿真警告示例.....45

表格目录

Table 3-1. 支持的逻辑操作..... 14 Table 3-2. 自定义波形代码指令..... 19 Table 3-3. SystemC 顶层跟踪信号..... 21 Table 3-4. 异步输出整形块跟踪信号（CLB Type 2）.....21 Table 3-5. 边界跟踪信号.....21 Table 3-6. 计数器块跟踪信号.....21 Table 3-7. 有限状态机块跟踪信号.....23 Table 3-8. 高级控制器块跟踪信号.....23 Table 3-9. 查找表块跟踪信号..... 24 Table 3-10. 输出查找表块跟踪信号..... 24 Table 4-1. 示例 1：工作模式..... 28 Table 4-2. PWM 输出..... 29 Table 4-3. 示例 4：信号连接..... 33

商标

C2000™ 和 Code Composer Studio™ 是 Texas Instruments 的商标。所有商标均属于其各自所有者。

Copyright © 2023 Texas Instruments Incorporated

1 引言

CLB 是集成在部分 C2000™ 器件中的硬件模块。CLB 包含一组可配置逻辑块和互连资源，用户可据此创建自定义数字逻辑，其实现方式类似于 FPGA。CLB 可扩展现有器件外设的功能，也可实现新的外设功能。过去已针对不同器件推出多个版本的 CLB。有关特定器件支持的 CLB 类型，请参阅 *C2000 Real-Time Control MCU Peripherals*。本文将用于配置 CLB 的软件实用程序称为 CLB 工具。

1.1 CLB 工具概述

CLB 工具可对每个 CLB 逻辑单元（tile）中的子模块进行配置和互连。该工具在 SysConfig 图形用户界面（GUI）中以 Tile Design 名称提供。SysConfig 是 Code Composer Studio™（CCS）的一部分。该工具还提供示例，供用户了解其功能并创建自己的项目。

该工具会生成一个 C 头文件，其中包含与用户在 GUI 中定义的配置设置对应的一组常量。工具还会生成一个 C 源文件，该文件通过一系列寄存器加载操作，将这些常量写入 CLB 寄存器，以此初始化 CLB 模块。设备初始化期间必须调用该 C 源文件中的函数。若要配置 CLB 逻辑单元（tile）与其他器件外设（包括交叉开关（XBAR）和其他 CLB 逻辑单元（tile））之间的输入、输出连接，请使用 SysConfig 中的 CLB 模块。此类连接需在 CLB 工具之外单独配置。CLB 工具还支持仿真这些输入，以测试逻辑单元配置的功能。

1.2 CLB 配置流程概述

CLB 工具基于 CCS 中的 SysConfig 工具。结合 C2000Ware SDK，工具生成的 C 头文件和源文件可用于配置 CLB 。若要仿真设计，还需安装若干第三方工具，包括编译器和波形查看器。有关 CLB 仿真器的详细信息，请参阅第 3.5 节。

CLB 工具会生成“.dot”文件，以图示方式展示子模块之间的连接，帮助检查设计。CCS 的构建后步骤会使用随示例提供的 node.js 和 JavaScript 库生成 HTML 和 SVG 格式的图示。工具还会生成“clb_sim.cpp”文件。该 CPP 文件与其他 CLB 仿真模型一起使用 GCC 编译器编译，生成的“.exe”文件必须在本地计算机上运行，以生成“.vcd”文件。随后可使用外部图形查看器对该“.vcd”文件进行时序分析。以上步骤均由工具生成的“clb_simulation”文件执行。根据所用操作系统，该文件为批处理文件（.bat）或 shell 脚本（.sh）（本用户指南中的图示以 Windows 操作系统为例）。必须执行此文件才能生成相应的“.vcd”文件。

CLB 配置编码在生成的 C 头文件“clb_config.h”中。CLB 工具生成的“clb_config.c”文件使用该头文件，将配置加载到 CLB 模块的寄存器中。必须在 SysConfig 中的 CLB 模块上关联相应的 Tile Design，配置才会生效，如图 1-1 所示。SysConfig 中的 Tile Design 与 CLB 模块彼此独立，因此即使器件上的 CLB 实例数量有限，也可以创建任意数量的 tile 配置，再按需将这些 Tile Design 加载到 CLB 中。

Copyright © 2023 Texas Instruments Incorporated

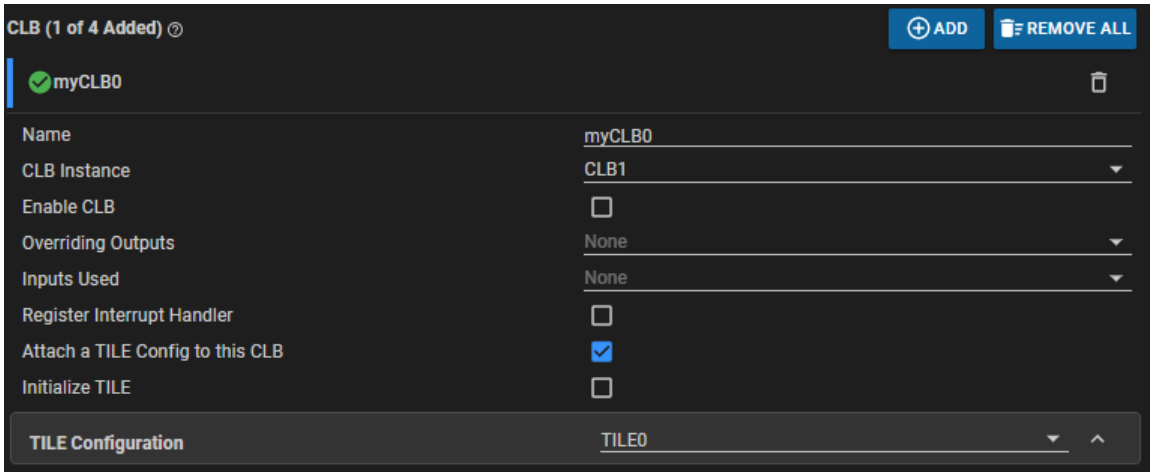


图 1-1. 将 tile 设计附加到 CLB 实例

请注意，在 C28x 器件的应用程序代码中，必须在器件初始化过程中调用“clb_config.c”文件中的函数。图 1-2 展示了 CLB 工具的输出及构建后步骤。

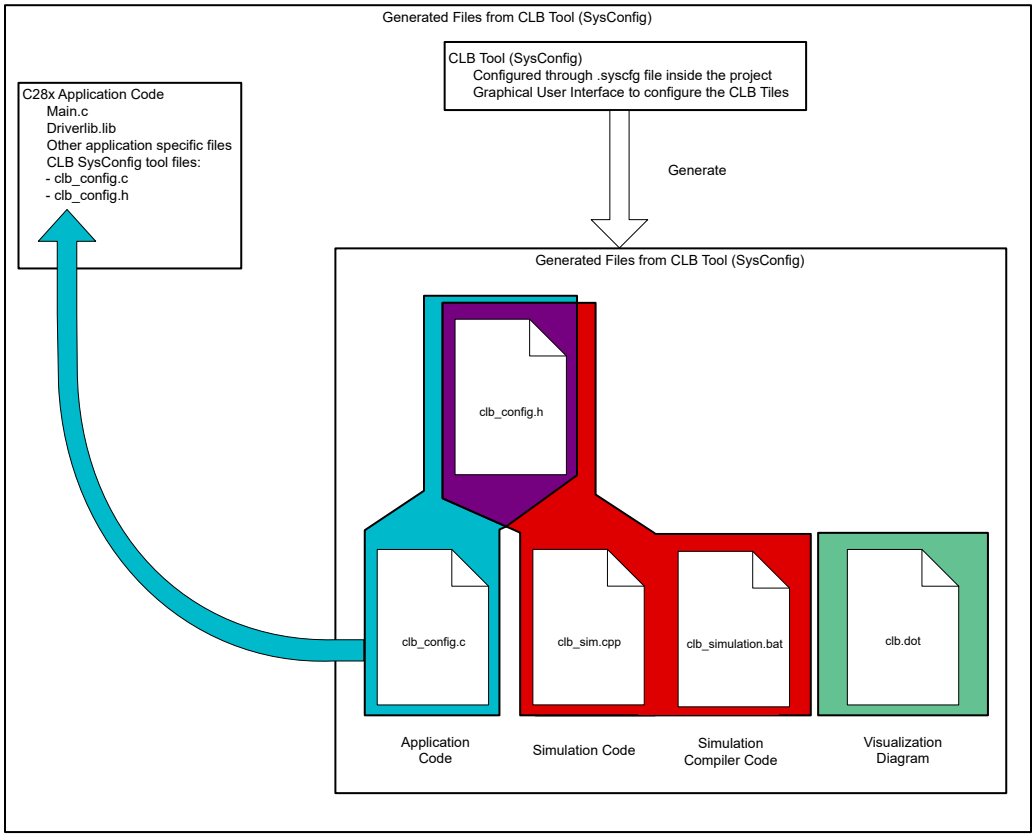


图 1-2. CLB 工具项目结构

通常，用户首先要定义 CLB 逻辑功能，可以通过逻辑电路图、时序信息、文字说明、VHDL 代码或其他形式进行描述。安装所需工具后，第一步是连接 CLB 逻辑单元（tile）中的子模块，以实现目标逻辑。

如果规格中包含时序图，用户可以仿真 CLB 配置，以确认其行为符合预期。这一步需要定义一组输入测试激励，并构建仿真项目来生成波形，然后在图形查看器中打开波形。如果结果不符合预期，可以修改 CLB 工具设置并重新仿真。

Copyright © 2023 Texas Instruments Incorporated

仿真得到正确波形后，可使用 CCS 将设计下载到器件，并运行或调试代码。

对于已为 C28x 器件启用 SysConfig 的 CCS 项目，生成 Tile Design 的 HTML 和 SVG 方框图属于构建后步骤的一部分。构建 CCS 项目时，应用程序代码以及生成的“clb_config.h”和“clb_config.c”文件会由 C28x 编译器编译，并生成“.out”文件。

“clb_simulation”文件使用 GCC 编译器编译生成的仿真文件“clb_sim.cpp”和“clb_config.h”，以及 CLB 仿真模型。此步骤会生成“.exe”文件（“simulation_output.exe”），随后自动运行该文件并生成“CLB.vcd”。可使用外部图形查看器查看此文件。

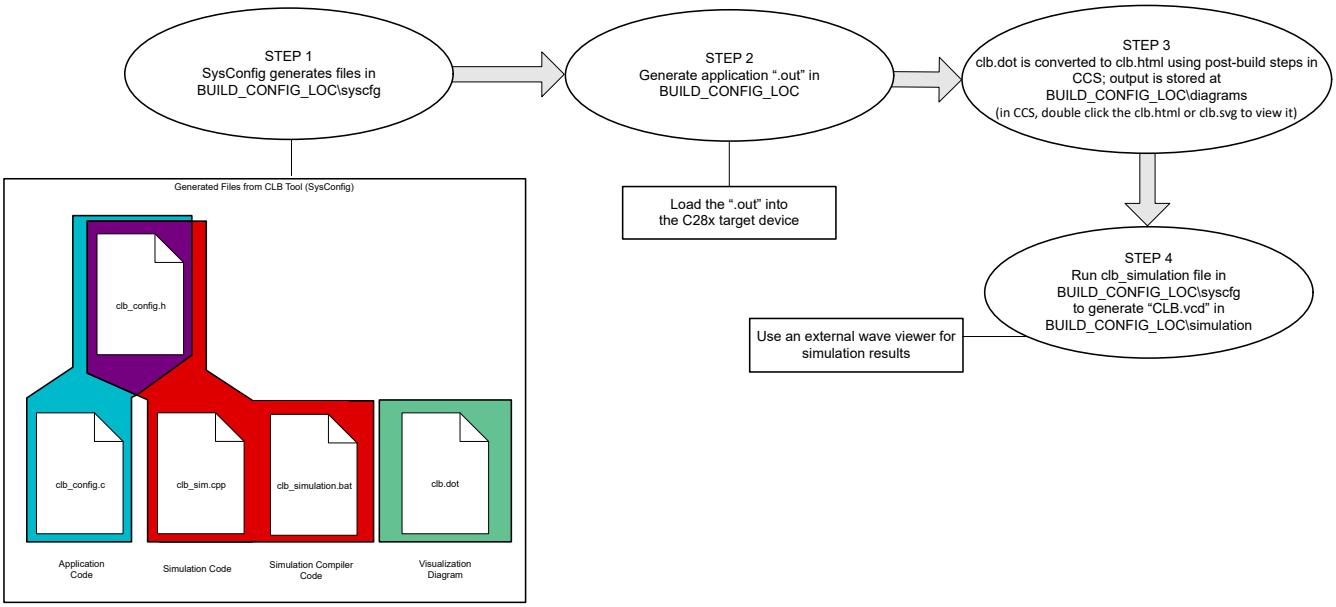


图 1-3. CLB 工具构建流程

在 CLB 工具构建流程中，BUILD_CONFIG_LOC 目录是项目的构建配置输出目录（例如“Debug”或“CPU1_RAM”），其名称与当前活动的构建配置相同。

2 入门

本节旨在帮助新用户快速上手 CLB 工具。

2.1 CLB 相关资料

基础资料

- C2000 Academy - CLB
- C2000™ 可配置逻辑块（CLB）系列（视频）
- 定制片上外设，突破传统逻辑设计的限制
- *Enable Differentiation and win with CLB in various applications Application Report*
- 在多种汽车应用中借助可配置逻辑实现差异化（视频）

入门资料

- *C2000™ Position Manager PTO API Reference Guide Application Report*
- *CLB Tool User Guide*
  – 基础示例为 7 - 15，建议从这些示例开始。更复杂的示例为 1-6。
- *Designing With The C2000 Configurable Logic Block Application Report*
- 如何为现有的 DriverLib 项目添加 SYSCONFIG 支持（引脚复用和外设初始化）？
- *How to Migrate Custom Logic From an FPGA/CPLD to C2000 Microcontrollers Application Report*

Copyright © 2023 Texas Instruments Incorporated

– 第 1-3 章非常适合入门和学习 CLB；后续章节属于专家资料，适合将 FPGA/CPLD 迁移到 C2000 CLB 的读者。

专家资料

- *Achieve Delayed Protection for Three-Level Inverter With CLB Application Report*
- *Diagnosing Delta-Sigma Modulator Bitstream Using C2000™ Configurable Logic Block Application Report*
- *How to Implement Custom Serial Interfaces Using Configurable Logic Block (CLB) Application Report*
- *Tamagawa T-Format Absolute-Encoder Master Interface Reference Design for C2000™ MCUs*

2.2 引言

若要使用该工具，必须安装 CCS 9.0 或更高版本。较早版本的 CCS 不包含配置 CLB 所需的 SysConfig 实用程序。有关 CCS 的更多信息和下载选项，请访问
CCS 下载页面。

上述工具可用于配置 CLB。若要仿真设计，还必须安装以下第三方（非 TI）工具：

- GNU 编译器（TDM-GCC）
- 仿真波形查看器（GTK Wave）

2.3 安装

2.3.1 安装编译 SystemC 所需的工具

要在 Windows 上编译仿真源文件 clb_sim.cpp，请按以下步骤操作：

1. 从 SourceForge 下载版本为 5.1.0-2 的“tdm-gcc”。2. 打开下载的安装文件。3. 取消选中“Check for updated files on the TDM-GCC server”（检查 TDM-GCC 服务器上是否有更新文件）选项。4. 在安装向导中选择“Create”（创建）。

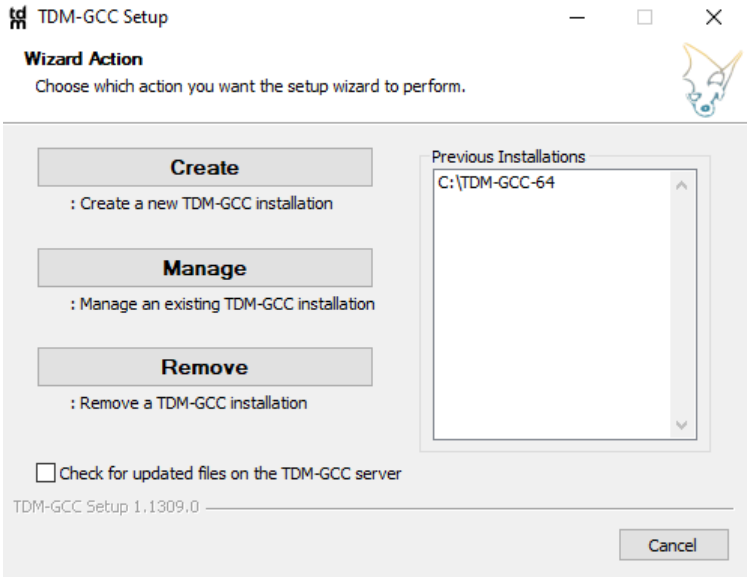


图 2-1. TDM 编译器安装向导

Copyright © 2023 Texas Instruments Incorporated

5. 如果向导显示有关许可证变更的信息，请选择“Next”（下一步）。

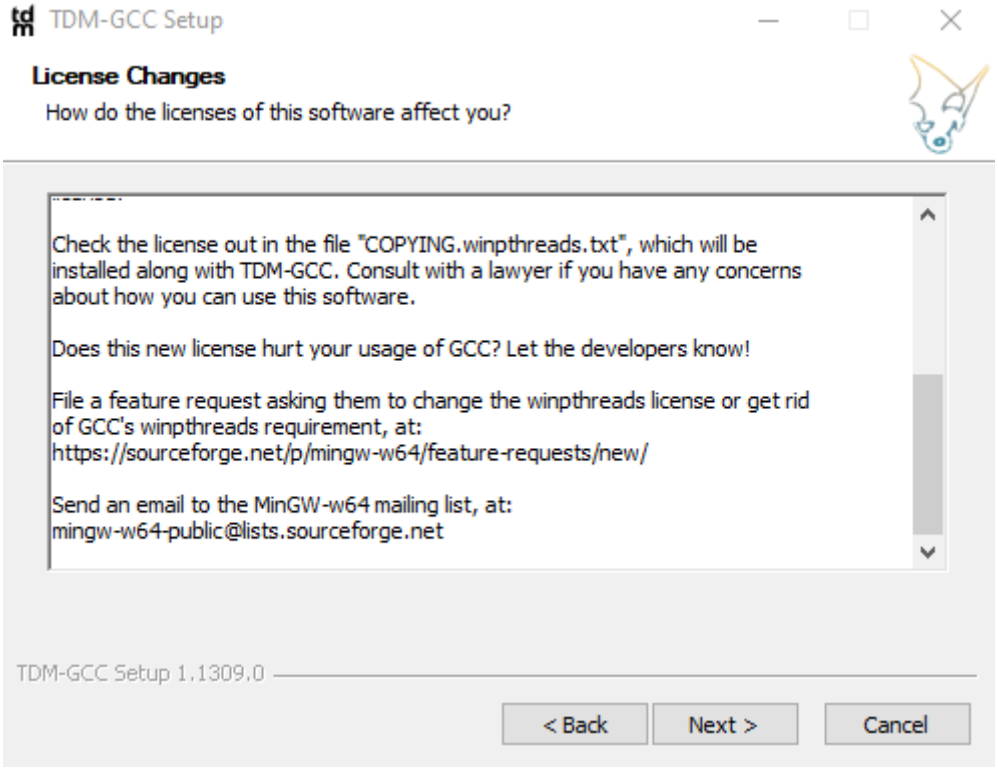


图 2-2. 更改 TDM 许可证

6. 将安装目录设为 C:\TDM-GCC-64，然后单击“Next”（下一步）。

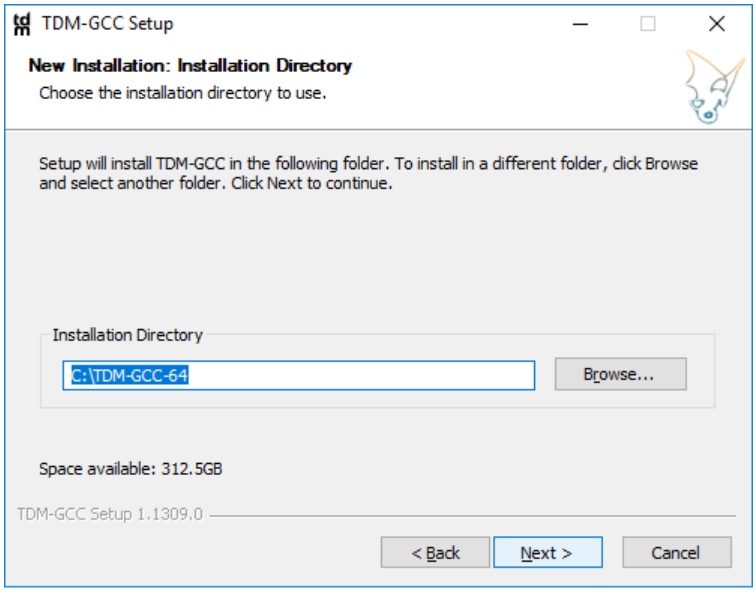


图 2-3. TDM 编译器路径

Copyright © 2023 Texas Instruments Incorporated

7. 单击“Install”（安装）之前，请确认已选中正确的组件。

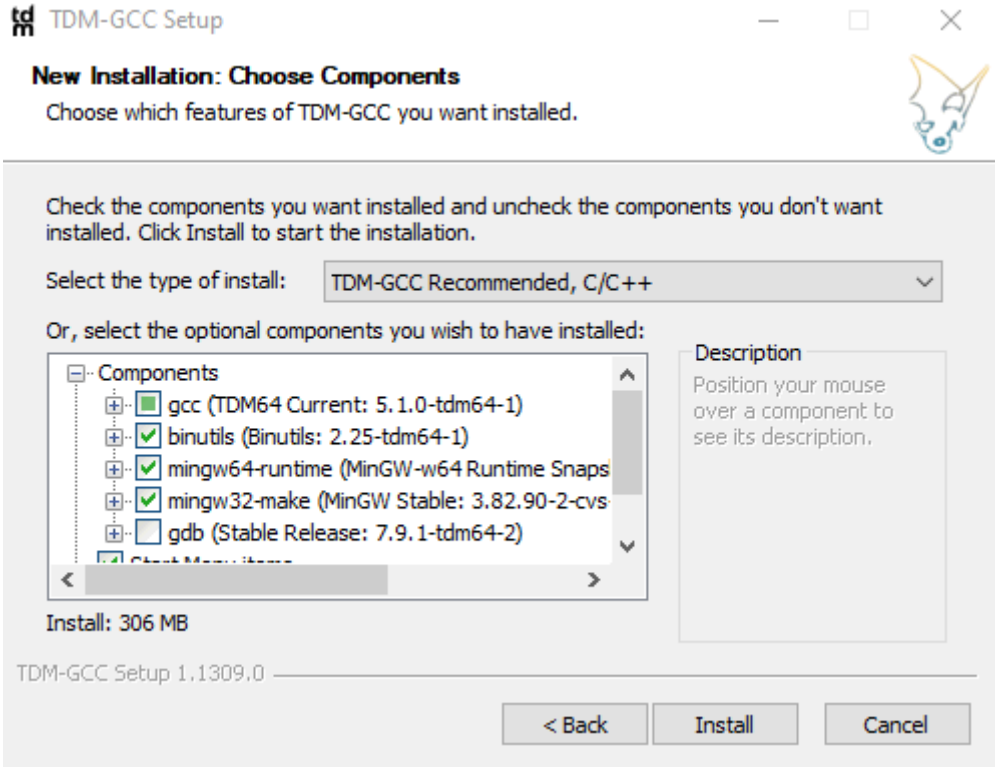


图 2-4. TDM 组件

8. 安装完成后，依次选择“Next”（下一步）和“Finish”（完成）。

在 Mac 或 Linux 上，需要安装 SystemC 库，但无需安装 G++ 编译器。继续之前，请确认 G++ 编译器为最新版本。若要在 Mac 或 Linux 上安装 SystemC，请按以下步骤操作：

1. 打开终端。2. 运行 sudo apt-get install build-essential。3. 从 Accelera 安装 SystemC 2.3.3，并在终端中运行 tar -xvf systemc-2.3.3.tar.gz 解压。4. 运行 sudo cp -r systemc-2.3.3 /usr/bin，将解压后的文件夹复制到“/usr/bin”目录。

进入 tar 命令创建的目录（不要进入“/usr/bin”），并创建名为“objdir”的目录。

```
~/Downloads$ sudo cp -r systemc-2.3.3 /usr/bin
~/Downloads$ cd systemc-2.3.3
~/Downloads/systemc-2.3.3$ mkdir objdir
~/Downloads/systemc-2.3.3$ cd objdir
```

图 2-5. 创建 SystemC 目录

Copyright © 2023 Texas Instruments Incorporated

5. 运行 sudo ../configure --prefix=/usr/bin/systemc-2.3.3/.

```
-----------------------------------------------------------------------
Configuration summary of SystemC 2.3.3 for x86_64-unknown-linux-gnu
-----------------------------------------------------------------------

Directory setup (based on classic layout):
  Installation prefix (aka SYSTEMC_HOME):
     /usr/bin/systemc-2.3.3
  Header files  : <SYSTEMC_HOME>/include
  Libraries     : <SYSTEMC_HOME>/lib-linux64
  Documentation : <SYSTEMC_HOME>/docs
  Examples      : <SYSTEMC_HOME>/examples

Architecture    : linux64
Compiler        : g++ (C/C++)

Build settings:
  Enable compiler optimizations  : yes
  Include debugging symbols      : no
  Coroutine package for processes: QuickThreads
  Enable VCD scopes by default   : yes
  Disable async_request_update   : no
  Phase callbacks (experimental) : no

-----------------------------------------------------------------------
```

图 2-6. SystemC 配置输出

6. 运行 sudo make。

```
  CXX      tracing/sc_wif_trace.lo
  CXX      utils/sc_hash.lo
  CXX      utils/sc_list.lo
  CXX      utils/sc_mempool.lo
  CXX      utils/sc_pq.lo
  CXX      utils/sc_report.lo
  CXX      utils/sc_report_handler.lo
  CXX      utils/sc_stop_here.lo
  CXX      utils/sc_string.lo
  CXX      utils/sc_utils_ids.lo
  CXX      utils/sc_vector.lo
  CXXLD    libsysc.la
make[3]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src/sysc'
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src/sysc'
Making all in tlm_core
make[2]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src/tlm_core'
  CXX      tlm_2/tlm_generic_payload/tlm_gp.lo
  CXX      tlm_2/tlm_generic_payload/tlm_phase.lo
  CXX      tlm_2/tlm_quantum/tlm_global_quantum.lo
  CXXLD    libtlm_core.la
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src/tlm_core'
Making all in tlm_utils
make[2]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src/tlm_utils'
  CXX      convenience_socket_bases.lo
  CXX      instance_specific_extensions.lo
  CXXLD    libtlm_utils.la
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src/tlm_utils'
Making all in .
make[2]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src'
  CXXLD    libsystemc.la
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src'
make[1]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/src'
Making all in examples
make[1]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples'
Making all in sysc
make[2]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/sysc'
  GEN      copy-check-data
To compile and run the examples type
   make check
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/sysc'
Making all in tlm
make[2]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/tlm'
Making all in common
make[3]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/tlm/common'
  GEN      copy-check-data
To compile the TLM examples library type
   make check
make[3]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/tlm/common'
Making all in .
make[3]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/tlm'
  GEN      copy-check-data
make[3]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/tlm'
To compile and run the examples type
   make check
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples/tlm'
make[2]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples'
  GEN      copy-check-data
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples'
make[1]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples'
make[1]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir'
make[1]: Nothing to be done for 'all-am'.
make[1]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir'
                ~/Downloads/systemc-2.3.3/objdir$
```

图 2-7. Make 输出

Copyright © 2023 Texas Instruments Incorporated

7. 运行 sudo make install。

```
make[3]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples'
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples'
make[1]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir/examples'
make[1]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir'
make[2]: Entering directory '/home/a0498187/Downloads/systemc-2.3.3/objdir'
make[2]: Nothing to be done for 'install-exec-am'.
 /bin/mkdir -p '/usr/bin/systemc-2.3.3'
 /usr/bin/install -c -m 644 ../AUTHORS ../NOTICE ../ChangeLog ../INSTALL ../LICENSE ../NEWS ../README ../RELEASENOTES ../cmake/INSTALL_USING_CMAKE '/usr/bin/systemc-2.3.3'
make[2]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir'
make[1]: Leaving directory '/home/a0498187/Downloads/systemc-2.3.3/objdir'
                ~/Downloads/systemc-2.3.3/objdir$
```

图 2-8. Make install 输出

2.3.2 安装仿真查看器

在 Windows 上安装 GTKWave：

1. 从 SourceForge 下载波形查看器 GTKwave。2. 下载适用于当前 Windows 版本的本机二进制文件（64 位 Windows 请选择“gtkwave-3.3.100-bin-win64”），并将下载的 ZIP 文件解压到 c:\gtkwave 目录。

在 Mac 或 Linux 上安装 GTKWave：
1. 在终端中运行命令 sudo apt install gtkwave。

3 使用 CLB 工具

本节介绍如何使用 CLB 工具配置 CLB 逻辑单元（tile）。CLB 工具要求 CCS 版本为 9.0 或更高版本。

3.1 导入空白 CLB 项目

启用了 CLB 的 DriverLib 示例项目位于 <C2000WARE_INSTALL>/driverlib/ <device>/examples。

例如，对于 F2837xD 器件，CLB 示例项目的路径为 <C2000WARE_INSTALL>/ driverlib/f2837xd/examples/ cpu1/clb。

1. 在 CCS 菜单中，选择“Project > Import CCS Projects…”。2. 在“Select search-directory”（选择搜索目录）中输入 CLB 示例项目的路径并单击“Refresh”（刷新）；也可以单击“Browse”（浏览）按钮，直接选择正确的文件夹。3. 选择“clb_empty”项目。4. 勾选“Copy projects into workspace”（将项目复制到工作区）。5. 单击“Finish”（完成）。

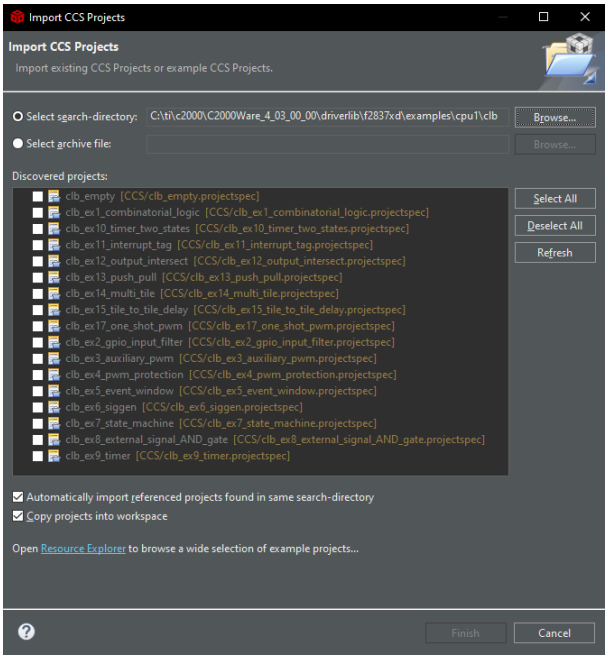


图 3-1. 导入 CCS Eclipse 项目

Copyright © 2023 Texas Instruments Incorporated

3.2 更新变量路径

上文导入的空白 CLB 项目不仅能够为 C28x 目标
器件生成“.out”文件，还能够生成仿真文件以及设计的 HTML/SVG 方框图。若要通过构建后步骤生成图示
，必须正确设置 C2000Ware 根目录和 Node 工具的路径。请按以下步骤再次确认这些路径：

1. 在项目上单击鼠标右键，然后选择“Project Properties”（项目属性）。2. 在“Resources”
（资源）下选择“Linked Resources”（链接资源）。3. 确认以下路径正确：a. C2000WARE_R
OOT（用于 CLB 图示及其他包含路径）

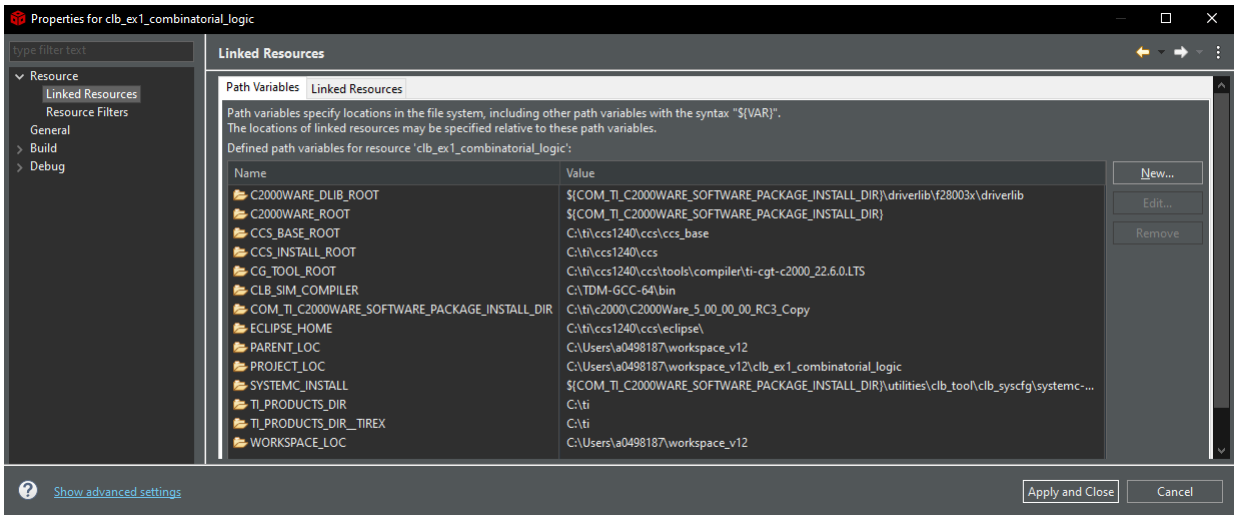


图 3-2. 链接资源

4. 如果名称左侧的图标不是文件夹，而是感叹号，表示系统中不存在该路径，必须手动选择正确路径。5. 确认以下系统变量的路径正确：a. NODE_TOOL

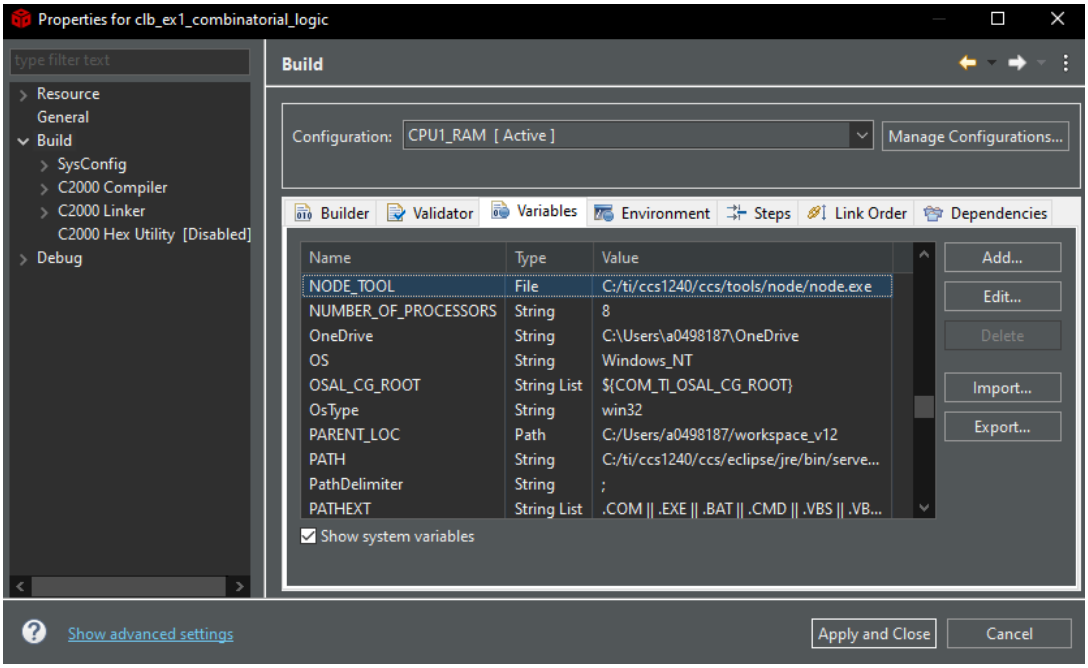


图 3-3. 构建变量

Copyright © 2023 Texas Instruments Incorporated

若要生成方框图，必须将构建变量 GENERATE_DIAGRAM 设为 1。可在“Project Properties”>“Build”>“Variables”中查看项目的构建变量，如图 3-4 所示。此变量会启用构建完成后在“Steps”下列出的步骤。方框图位于项目相应构建配置的“diagrams”目录中，如图 5-5 所示。

Note

For Mac and Linux, the conditionals used in the post-build steps will not be able to execute properly. To generate the diagram, the test `if ${GENERATE_DIAGRAM} == 1` must be removed.

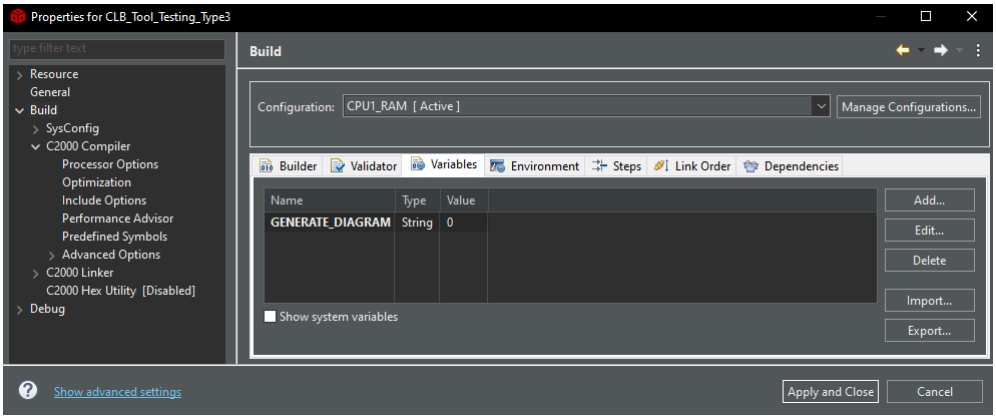


图 3-4. 用于生成图示的构建变量

3.3 配置 CLB 逻辑单元（tile）

若要打开 SysConfig 工具，请在 CCS 的 Project Explorer（项目资源管理器）窗口中双击要编辑的“.syscfg”文件。界面示例如图 3-5 所示。

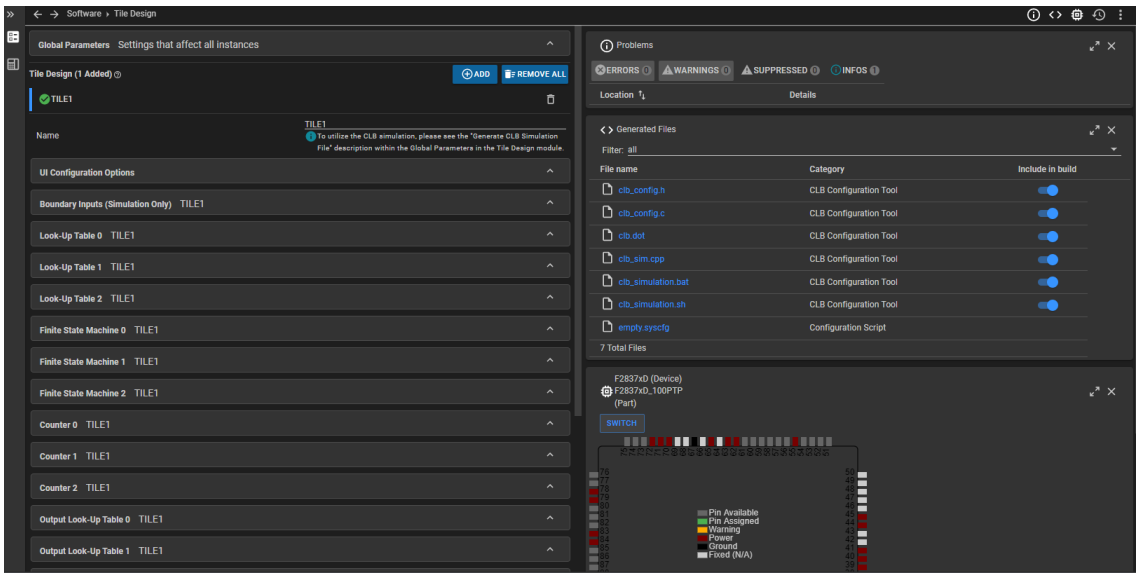


图 3-5. CLB 工具 SysConfig 界面

如果该界面未打开，请确认之前的步骤均已正确完成。

CLB 逻辑单元的配置保存在 .syscfg 文件中。需要时可以更改 tile 的名称。对于当前选中的 tile，右侧窗格会列出其子模块。选中某个子模块后，即可查看和编辑其参数。

Copyright © 2023 Texas Instruments Incorporated

“BOUNDARY”项是一个特殊情况。该组仅用于选择仿真时使用的逻辑单元输入。生成工具配置时，CLB 输入始终来自 SysConfig 中的 CLB 模块；而在仿真时，用户可以指定方波信号源及其周期和占空比（单位均为时钟周期），并设置同步方式和输入流水线条件，如图 3-6 所示。仿真还支持生成自定义波形。有关仿真器的更多信息，请参阅第 3.5 节。这些选项仅用于仿真，不会影响实际 CLB 配置或其在器件上的实现。

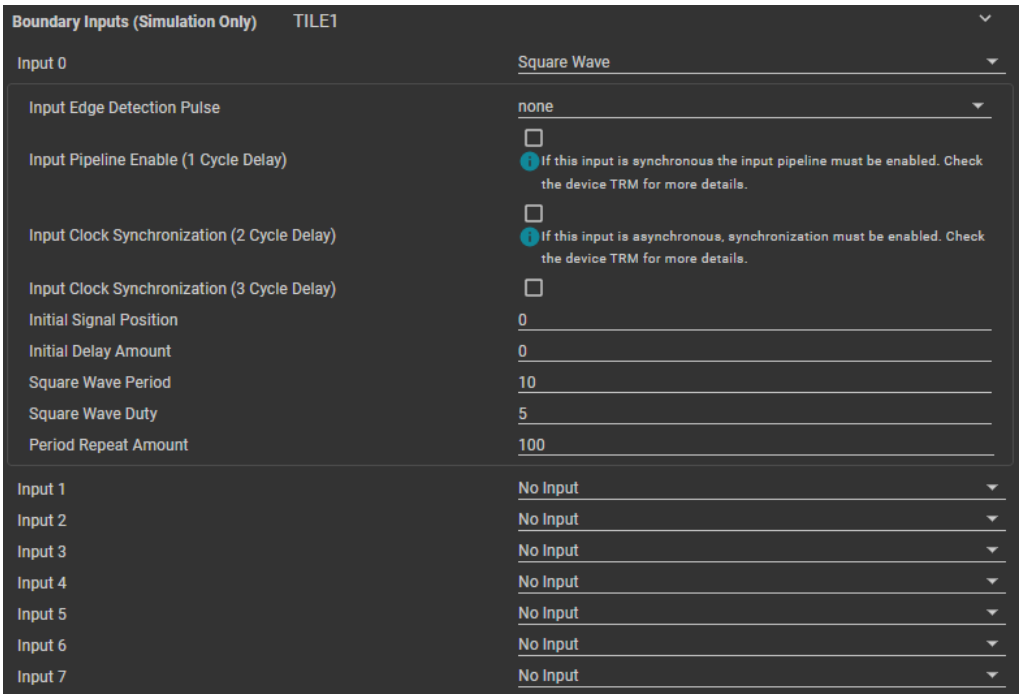


图 3-6. 边界输入选项

用户可通过工具中的复选框和下拉选项，配置并连接每个 CLB 逻辑单元（tile）中的子模块。除“BOUNDARY”外，每个子模块都带有“User Description”（用户说明）。这是一个多行文本框，用户可在其中填写注释，说明 CLB 各部分的用途和背景。

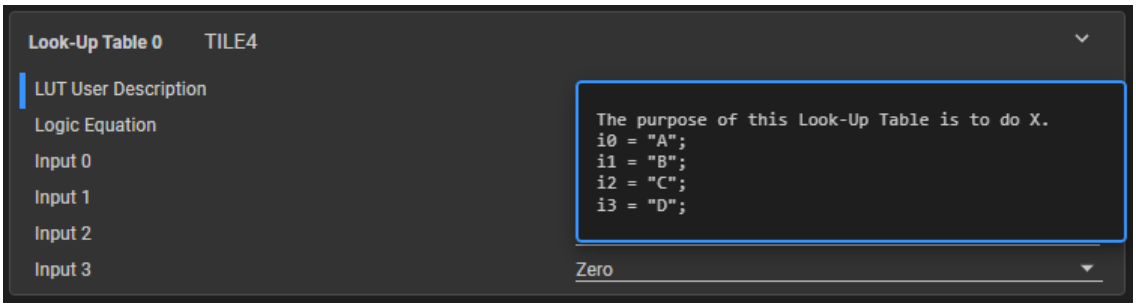


图 3-7. 用户说明文本框

Copyright © 2023 Texas Instruments Incorporated

将鼠标悬停在配置工具中的各项上时，会显示上下文相关帮助。图 3-8 展示了“Counter 0”子模块中“Match Reference 1”字段的示例。

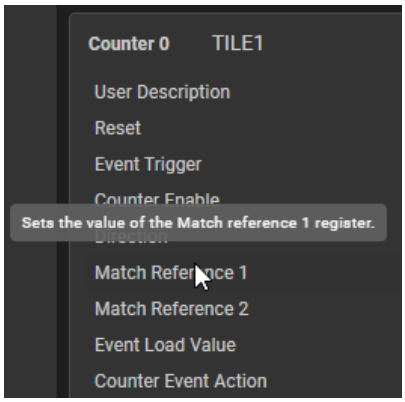


图 3-8. 计数器选项

LUT 和 FSM 的逻辑方程通过文本输入配置，格式采用 C 语言语法。表 3-1 列出了布尔方程中允许使用的符号。

表 3-1. 支持的逻辑操作

Logical Operation	Symbol
AND	&
OR	|
XOR	^
NOT	!

支持使用括号，例如可以写为：i1 | !(i2 & i3)。工具会在输入方程时进行语法检查。若方程无效，输入行下方会显示错误消息。

某些不常见的逻辑组合会触发警告。图 3-9 展示了一个示例：用户在“LUT 0”的布尔方程中尝试使用输入 i2，但 i2 被配置为常量，这可能并非用户本意。方程下方和输入选择项下方都会显示警告。

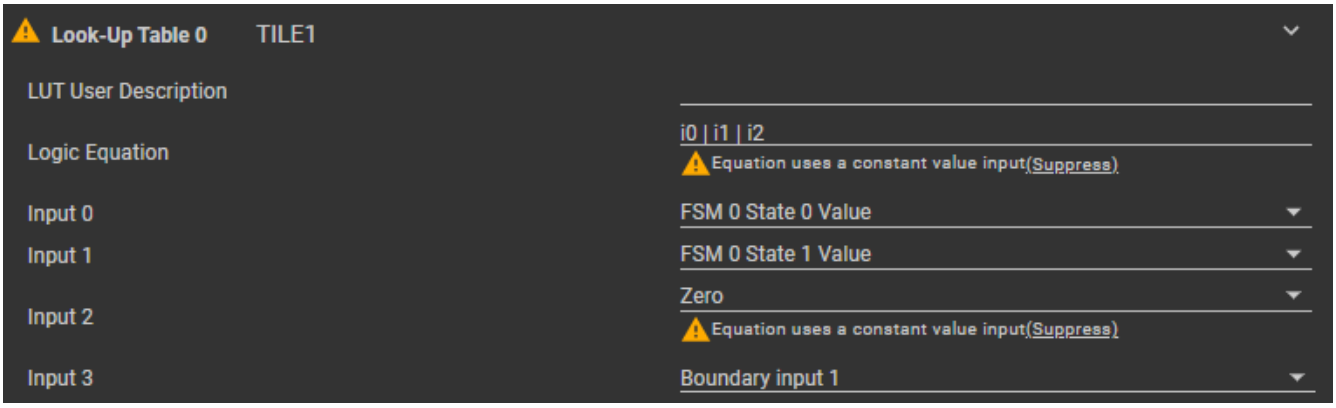


图 3-9. 方程警告

对于某些字段，工具会检查数值输入是否在允许范围内。例如，若尝试将大于 2^{32} 的值加载到计数器子模块中，工具会发出警告，因为该计数器宽度仅为 32 位。

用户输入配置数据时，工具会自动生成多个文件。若要查看生成的文件
文件，请单击工具右上角的“< >”图标，并选择要打开的文件名。

Copyright © 2023 Texas Instruments Incorporated

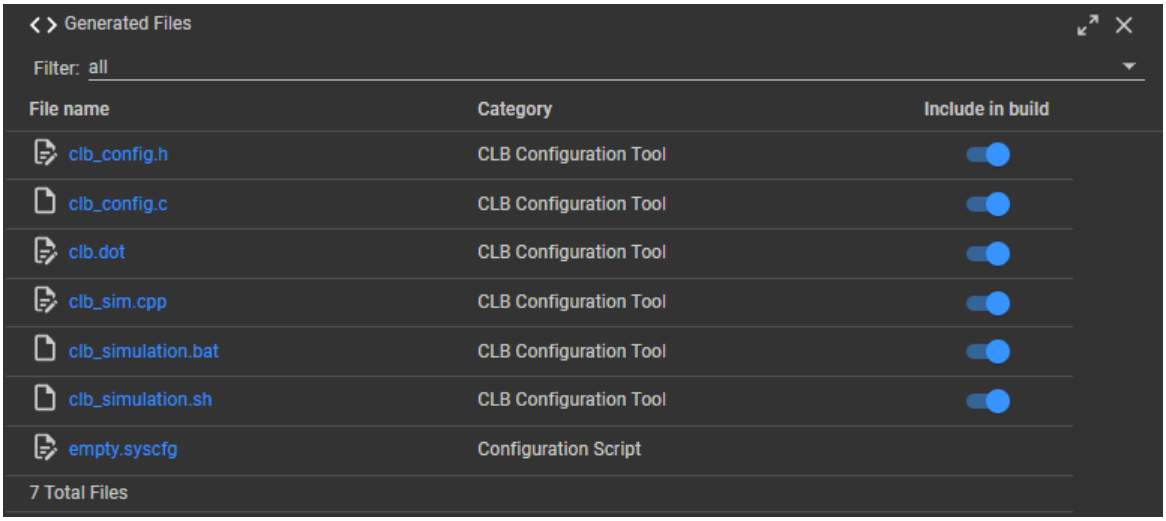


图 3-10. CLB 工具生成的文件

CLB 寄存器设置保存在头文件“clb_config.h”中，用户可以单击文件名打开它。示例如图 3-11 所示。请注意，每次用户更改 CLB Tile Design 设置时，工具都会更新该文件。因此，手动修改生成文件的内容会被工具覆盖。如果在更改 CLB 设置时保持文件打开，选择“Unified Diff”（统一差异）选项后，即可查看文件中受影响寄存器数据的变化。

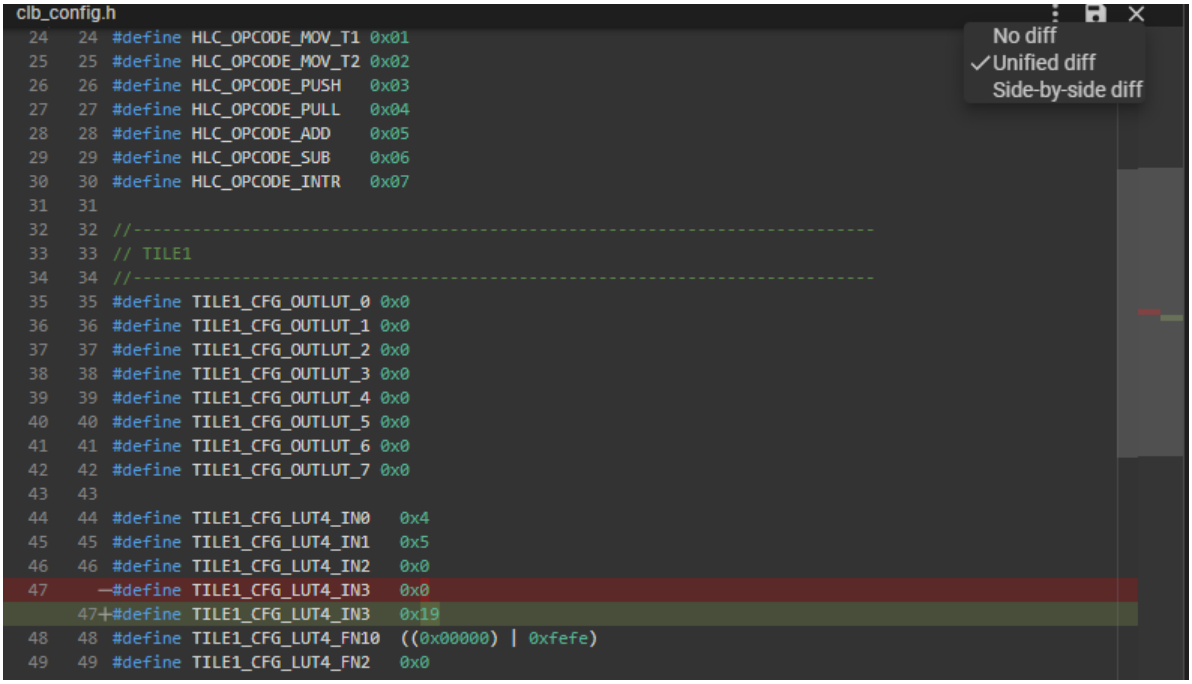


图 3-11. “clb.h”头文件示例

用户可通过“clb.dot”文件查看子模块互连关系的图形表示。构建后步骤会生成该方框图的 HTML 和 SVG 版本，可在 CCS 中打开和查看。

Copyright © 2023 Texas Instruments Incorporated

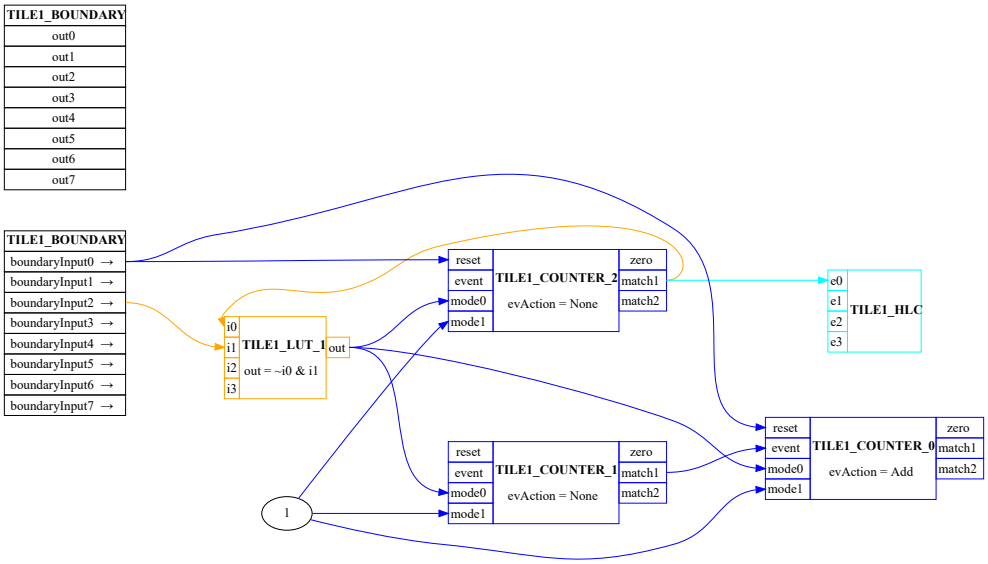


图 3-12. CLB 方框图

HLC 子模块的字段包括事件和初始值配置。4 个事件中的每个事件都可触发一个简短程序，最多包含 8 条指令。有关 HLC 的详细信息，请参阅特定器件的 TRM。

可在“Other Dependencies”（其他依赖项）下拉菜单的“HLC Program”中输入 HLC 指令。在用完全部 8 条指令之前，界面始终显示一个空白行。图 3-13 展示了一个示例：用户选择了一个 HLC 触发事件，并输入了由 3 条指令组成的简短程序。

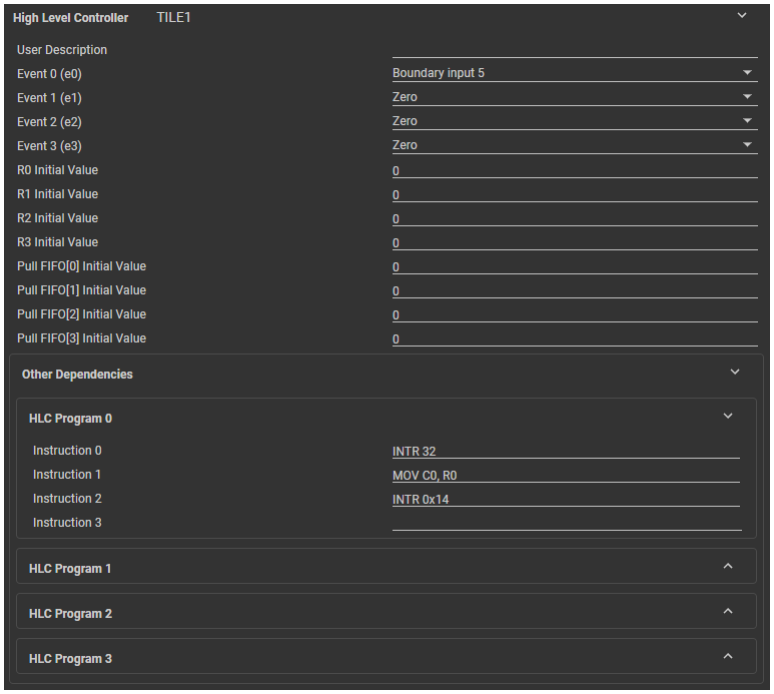


图 3-13. HLC 配置示例

Copyright © 2023 Texas Instruments Incorporated

3.4 创建 CLB 方框图

按照以下步骤创建 CLB 方框图：

1. 在 Build 选项中将构建变量 GENERATE_DIAGRAM 设为 1。

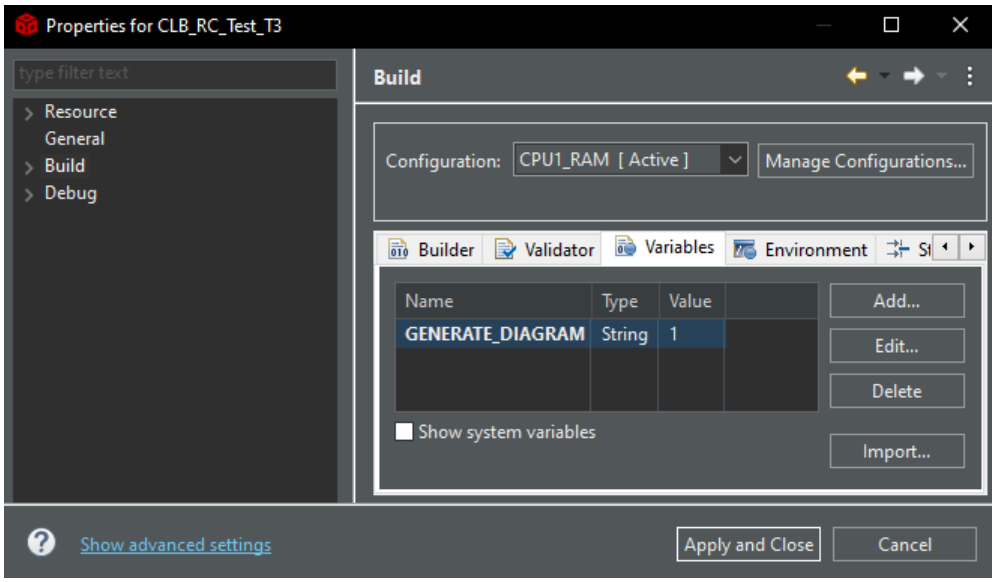


图 3-14. GENERATE_DIAGRAM 构建变量

注意

对于 Linux 或 Mac 等操作系统，为使其正常执行，必须从构建后步骤中移除条件语句 if ${GENERATE_DIAGRAM} == 1。有关 CLB 项目的构建后步骤示例，请参阅图 5-2。

2. 构建项目。3. 在 diagrams 目录中查看 HTML 或 SVG 图示，如图 5-5 所示。

3.5 使用仿真器

3.5.1 静态配置面板

配置工具顶部窗格包含仿真使用的“Global Parameters”（全局参数）设置。单击“?”图 标即可查看简要说明。

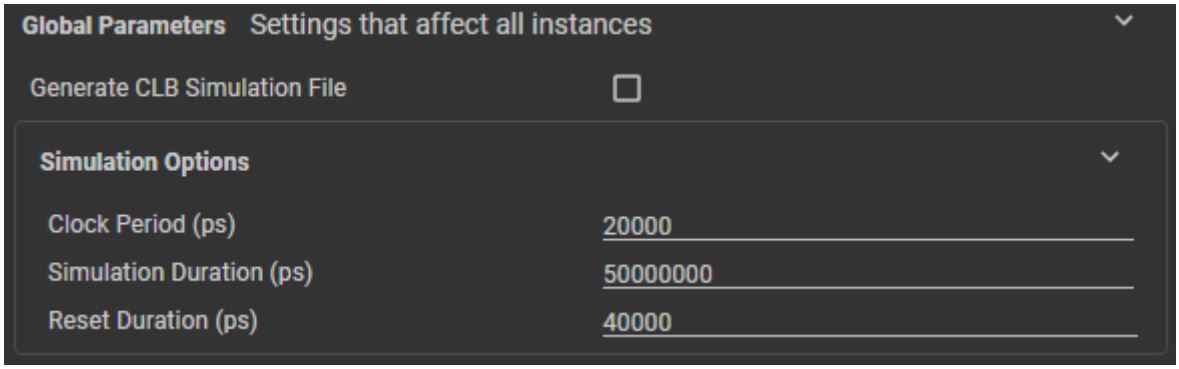


图 3-15. 静态选项

“Generate CLB Simulation File”用于生成“clb_simulation”文件的内容。执行该文件后，会为所有 CLB Tile Design 生成 .vcd 文件。该 .vcd 文件会出现在执行批处理文件的上一级目录中，并位于新建的“simulation”目录内。

Copyright © 2023 Texas Instruments Incorporated

“Clock Period (ps)”（时钟周期）指用于仿真的 CLB 时钟周期，单位为皮秒。“Simulation Duration (ps)”（仿真时长）字段用于设置仿真运行的持续时间，单位为皮秒。“Reset Duration (ps)”（复位时长）字段用于插入一段皮秒级延迟，以模拟器件复位的效果。

3.5.2 创建输入激励

在 CCS 的 Project Explorer（项目资源管理器）窗口中双击文件名，打开 .syscfg 文件。选中“Boundary”类别以展开它。

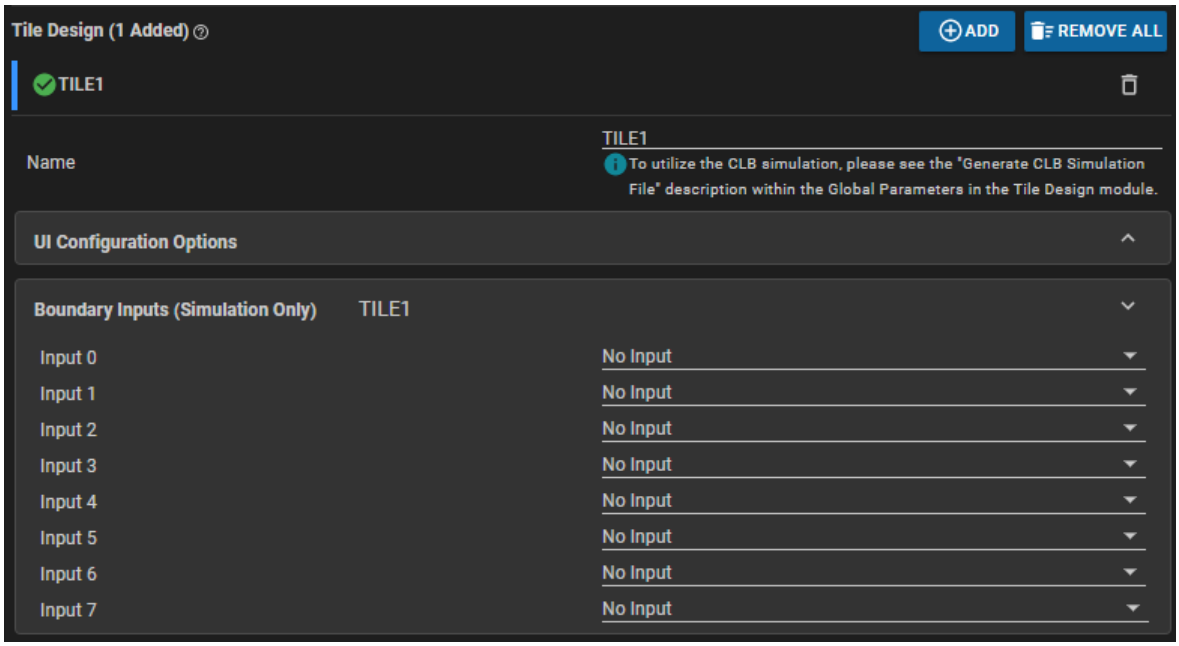


图 3-16. 边界输入 0 至 7

可通过下拉菜单为八个 CLB 输入中的每个输入分别定义激励信号。单击右侧的向下箭头以显示选项：

- 无输入（No Input）—默认选项，不生成激励信号。
- 方波（Square Wave）—定义周期性 PWM 输入，可配置初始信号电平、初始延迟和周期，duty, and period repeat amount.

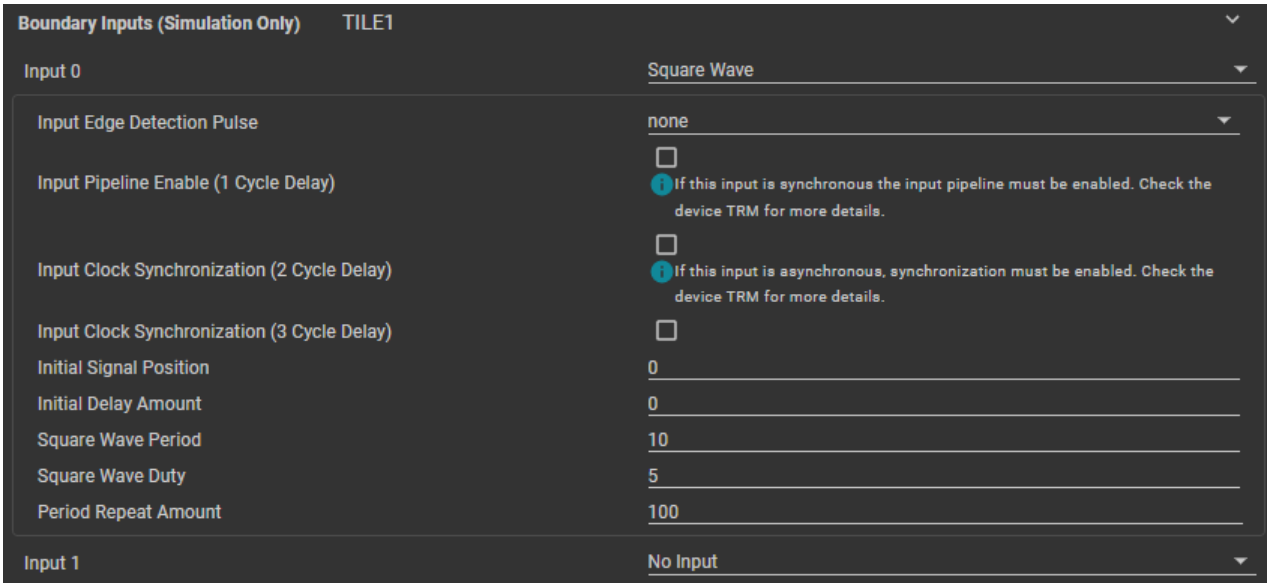


图 3-17. 边界输入方波

Copyright © 2023 Texas Instruments Incorporated

在图 3-17 中，PWM 波的周期和占空比分别设为 10 和 5 个 CLB 时钟周期。“Input Edge Detection Pulse”（输入边沿检测脉冲）选项可让用户选择在该 PWM 波的上升沿和/或下降沿生成脉冲。

“Input Pipeline Enable”（启用输入流水线）复选框会为输入信号增加一个周期的延迟，适用于路由至 CLB 输入的同步信号。请注意，流水线滤波器仅适用于某些 CLB 类型。更多信息请参阅特定器件 TRM 中的 CLB 输入多路复用器部分。

“Input Clock Synchronization”（输入时钟同步）复选框会强制输入波形与 CLB 时钟同步（同步器会产生 2-3 个周期的延迟，因此针对这两种时序都提供了相应复选框，因为确切延迟无法预先确定）。对于相对于 CLB 异步的输入信号，必须启用此选项。更多信息请参阅特定器件 TRM 中的 CLB 输入多路复用器部分。

- 低电平 (0) 或高电平 (1) — 分别将激励设为固定低电平或固定高电平
- 自定义波形输入（Custom Wave Input）—使用伪代码生成自定义激励信号。

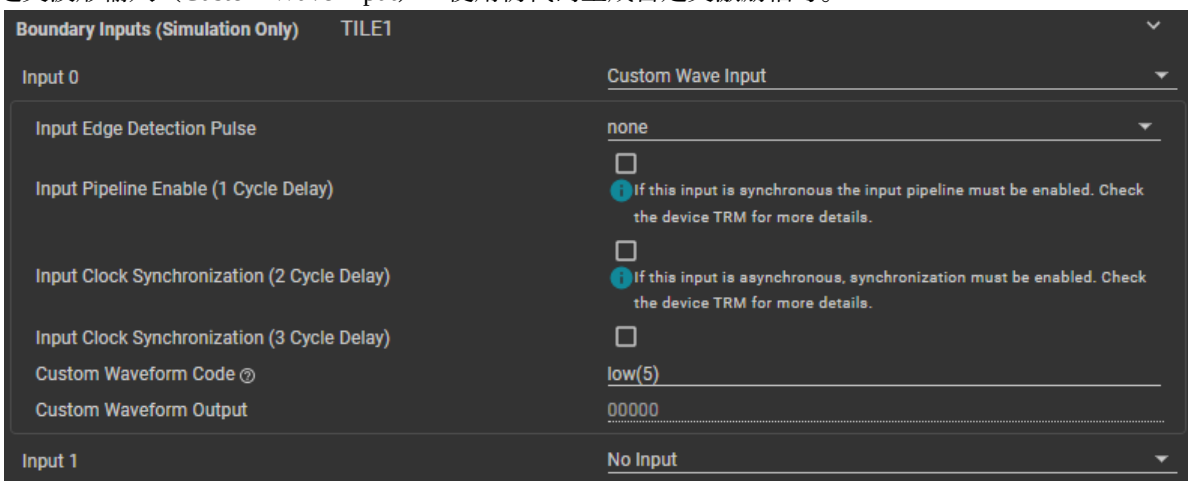


图 3-18. 自定义边界输入

“Input Edge Detection Pulse”、“Input Pipeline Enable”和“Input Clock Synchronization”的工作方式与方波激励相同，并支持使用自定义波形伪代码。对于“high”、“low”和“rpt”指令，其数值参数可使用十六进制（0x1A）或十进制（26）表示。

表 3-2. 自定义波形代码指令

Instruction	Description
#define	Pattern replacer used to define macros
high(N)	Sets waveform high for 'N' CLB cylces
low(N)	Sets waveform low for 'N' CLB cycles
rpt(N)	Starts a repeat block; code encapsulated with rpt(N) and rpt_end will be repeated a total of 'N' times
rpt_end	Denotes the end of a repeat block

Copyright © 2023 Texas Instruments Incorporated

- tile 输出（Tile Output）—使用所选 CLB 逻辑单元（tile）的输出来作为当前逻辑单元的输入激励。



图 3-19. 边界输入 tile 输出

- “Tile Name”必须是 CLB 工具项目中有效的 tile 名称。由于 CLB 逻辑单元（tile）的输出为同步信号，应启用“Input Pipeline Enable”。更多信息请参阅特定器件 TRM 中的 *CLB input mux* 部分。

3.5.3 运行仿真

定义好 CLB 配置和输入激励后，用户即可编译项目。以下列出完整 的生成“CLB.vcd”的步骤。

1. 在 Tile Design 模块顶部的“Global Parameters”下拉菜单中，勾选“Generate CLB Simulation File”。2. 构建项目，确认 Tile Design 设置没有错误。3. 在文件资源管理器或命令行中运行“clb_simulation”文件。该文件位于生成的 SysConfig 文件所在位置。对于 CCS，该位置是项目构建配置目录下的 syscfg 目录（例如“CPU1_RAM/syscfg”）。4. 打开“simulation”目录（位于运行“clb_simulation”的目录上一级）。5. 双击“CLB.vcd”文件。

如果波形查看器已配置完成，双击“CLB.vcd”文件
即可打开查看器并检查波形。图 3-20 展示了配置为显示输入波形示例的 GTKwave 查看器。有关如何添加和查看信号，请参阅查看器文档。

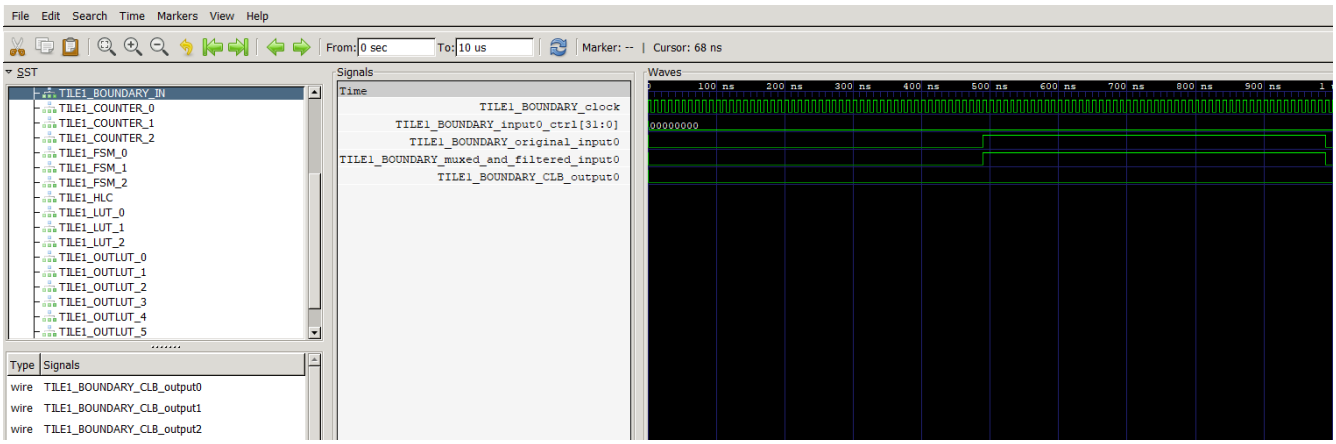


图 3-20. CLB 仿真示例

如果仿真波形与预期不符，请修改 .syscfg 文件中的配置并重新运行仿真。

3.5.4 跟踪信号说明

本节简要说明“CLB.vcd”文件中生成的各类跟踪信号。根据器件上 CLB 的类型不同，仿真中可能无法使用某些信号或信号组。

Copyright © 2023 Texas Instruments Incorporated

注意

以下约定用于缩写和简化跟踪信号列表：1. “TILE#”表示 Tile Design 实例的名称。2. “N”表示 0 至 7，用于表示所示信号的多个实例。3. “X”表示 0 至 2，用于表示所示信号的多个实例。4. “Y”表示 0 至 3，用于表示所示信号的多个实例。

表 3-3. SystemC 顶层跟踪信号

Trace Signal Name	Description
sc_top_clock	Clock signal of the CLB
sc_top_reset	Reset for the CLB
sc_top_enable	CLB enable signal; used to enable the CLB submodules

表 3-4. 异步输出整形块跟踪信号（CLB Type 2）

Trace Signal Name	Description
TILE#_AOC_N_clb_output	Output from the AOC submodule
TILE#_AOC_N_mux_ctrl [15:0]	AOC mux control value; value is in the same format as the CLB_OUTPUT_COND_CTRL_N register
TILE#_AOC_N_release_signal	Release signal; determines when the CLB output is set, cleared, or delayed depending on the release option chosen in TILE#_AOC_N_mux_ctrl (default release signal does not alter CLB output)
TILE#_AOC_N_gate_signal	Gate signal; this signal is logically combined with the CLB output using an AND, OR, or XOR depending on the selection chosen in TILE#_AOC_N_mux_ctrl (default gate signal does not alter output)
TILE#_AOC_N_mux_input_clb_tile_output	CLB output signal corresponding to the same AOC number of the signal; i.e. CLB output 0 is an input option for AOC 0
TILE#_AOC_N_mux_input_boundary_input	CLB boundary input signal corresponding to the same AOC number of the signal; i.e. CLB boundary input 0 is an input option for AOC 0

表 3-5. 边界跟踪信号

Trace Signal Name	Description
TILE#_BOUNDARY_CLB_outputN	CLB boundary output signal; this signal is routed out of the CLB peripheral to the rest of the device; this signal has passed through the AOC submodule if the device has this type of CLB (check the CLB Tile section in the device technical reference manual for more details)
TILE#_BOUNDARY_muxed_and_filtered_inputN	Input to the CLB module after passing through any synchronization, input pipeline filters, or edge filters enabled
TILE#_BOUNDARY_inputN_ctrl [31:0]	Control value for enabling synchronization, input pipeline filters, or edge filters
TILE#_BOUNDARY_original_inputN	Input to the CLB module before any modification such as synchronization, pipelining, or edge filtering
TILE#_BOUNDARY_clock	Clock signal of the CLB

表 3-6. 计数器块跟踪信号

Trace Signal Name	Description
TILE#_COUNTER_X_reconfig_pipeline_en	Enable for reconfigurable pipelining which pipelines the operations of the HLC and counter submodules (this is not the same enable as the pipeline input filter enable)
TILE#_COUNTER_X_counter_equals_match2	This signal is high when the counter value equals the match reference 2 value; if the match reference 2 tap output is enabled for serializer mode, this signal is high when the counter bit position selected is high
TILE#_COUNTER_X_counter_equals_match1	This signal is high when the counter value equals the match reference 1 value; if the match reference 1 tap output is enabled for serializer mode, this signal is high when the counter bit position selected is high

Copyright © 2023 Texas Instruments Incorporated

表 3-6. 计数器块跟踪信号（续）

Trace Signal Name	Description
TILE#_COUNTER_X_counter_equals_zero	This signal is high when the counter value equals zero
TILE#_COUNTER_X_match2_val [31:0]	Value of the match reference 2 value; can be modified by the HLC
TILE#_COUNTER_X_match1_val [31:0]	Value of the match reference 1 value; can be modified by the HLC
TILE#_COUNTER_X_counter_output [31:0]	Value of the counter itself
TILE#_COUNTER_X_hlc_match2_load_en	Load enable which determines when to load the match reference 2 value from the HLC submodule; this matches the TILE#_HLC_hlc_counterX_match2_load_en signal
TILE#_COUNTER_X_hlc_match1_load_en	Load enable which determines when to load the match reference 1 value from the HLC submodule; this matches the TILE#_HLC_hlc_counterX_match1_load_en signal
TILE#_COUNTER_X_hlc_counter_load_en	Load enable which determines when to load the counter with a value from the HLC submodule; this matches the TILE#_HLC_hlc_counterX_load_en signal
TILE#_COUNTER_X_hlc_counter_load_val [31:0]	Counter value loaded from the HLC submodule; this value is loaded to the match reference 1, match reference 2, or counter value depending on if the appropriate HLC load enable is set (i.e. depending on what instruction is being executed by the HLC). This matches the TILE#_HLC_counter_hlc_load_value signal
TILE#_COUNTER_X_match2_tap [4:0]	Specifies which bit of the counter to tap for match reference 2
TILE#_COUNTER_X_match2_tap_en	Enable which allows the counter to tap a bit specified by TILE#_COUNTER_X_match2_tap; TILE#_COUNTER_X_counter_equals_match2 is high when the appropriate tap bit is set, which effectively brings out a bit position from the counter to the match reference 2 output
TILE#_COUNTER_X_match1_tap [4:0]	Specifies which bit of the counter to tap for match reference 1
TILE#_COUNTER_X_match1_tap_en	Enable which allows the counter to tap a bit specified by TILE#_COUNTER_X_match1_tap; TILE#_COUNTER_X_counter_equals_match1 is high when the appropriate tap bit is set, which effectively brings out a bit position from the counter to the match reference 2 output
TILE#_COUNTER_X_lfsr_en	Enable for the Linear Feedback Shift Register; this allows the counter submodule to compute the CRC on a serial bit stream
TILE#_COUNTER_X_global_serializer_en	Enable for the serializer; when enabled, the counter loads either the next LFSR serial value or the appropriate serial value (see the device technical reference manual for more details)
TILE#_COUNTER_X_mode1	Controls the direction of the counter; counts up when set to 1, counts down when set to 0
TILE#_COUNTER_X_mode0	Controls whether the counter is stopped; enables counting when set to 1
TILE#_COUNTER_X_global_reset	Reset for the CLB
TILE#_COUNTER_X_add_or_shift_dir	When the counter event is enabled and not configured for a load event, this signal is high when the event adds or shifts the counter value left and is set 0 otherwise
TILE#_COUNTER_X_add_or_shift_on_event_en	This signal is set high if the counter is configured to add or shift when an event occurs and is set 0 otherwise
TILE#_COUNTER_X_add_or_shift_mode	This signal is set high if the counter is adding or subtracting and is set 0 otherwise
TILE#_COUNTER_X_global_en	CLB enable signal
TILE#_COUNTER_X_event_load_val [31:0]	When an event occurs and the event action is configured to be 'load', this value is loaded to the counter
TILE#_COUNTER_X_event	This signal is high when an event has occurred or is occurring
TILE#_COUNTER_X_counter_reset	Reset for the counter submodule
TILE#_COUNTER_X_clock	Clock signal of the CLB

Copyright © 2023 Texas Instruments Incorporated

表 3-7. 有限状态机块跟踪信号

Trace Signal Name	Description
TILE#_FSM_X_fsm_lut_output	Output for the FSM look-up table output equation
TILE#_FSM_X_fsm_s1_output	Output for the FSM state 1 equation
TILE#_FSM_X_fsm_s0_output	Output for the FSM state 0 equation
TILE#_FSM_X_LUT_output_equation [15:0]	Value representing the FSM look-up table output equation
TILE#_FSM_X_state1_equation_output [15:0]	Value representing the FSM state 1 equation
TILE#_FSM_X_state0_equation_output [15:0]	Value representing the FSM state 0 equation
TILE#_FSM_X_extra_external_input_select1	Selects where the value of external input 3 (e3) is coming from; when high e3 is used, and when low state 1 (s1) is used
TILE#_FSM_X_extra_external_input_select0	Selects where the value of external input 2 (e2) is coming from; when high e2 is used, and when low state 0 (s0) is used
TILE#_FSM_X_extra_external_input1	Extra external input 1 (xe1) to the FSM submodule; the signal chosen as xe1 comes from within the CLB and can be used only in the look-up table output equation
TILE#_FSM_X_extra_external_input0	Extra external input 0 (xe0) to the FSM submodule; the signal chosen as xe0 comes from within the CLB and can be used only in the look-up table output equation
TILE#_FSM_X_external_input1	External input 1 (e1) to the FSM submodule; the signal chosen as e1 comes from within the CLB and can be used in the state 0, state 1, and look-up table output equations
TILE#_FSM_X_external_input0	External input 0 (e0) to the FSM submodule; the signal chosen as e0 comes from within the CLB and can be used in the state 0, state 1, and look-up table output equations
TILE#_FSM_X_global_reset	Reset for the CLB
TILE#_FSM_X_global_en	CLB enable signal
TILE#_FSM_X_clock	Clock signal of the CLB

表 3-8. 高级控制器块跟踪信号

Trace Signal Name	Description
TILE#_HLC_spi_export_receive_buffer [15:0]	Representation of the data stored in a SPI RX buffer; this helps validate data exporting through the SPI buffer using the HLC submodule
TILE#_HLC_fifo_overflow_signal	When high, this signal indicates a FIFO overflow has occurred (pushing when the FIFO is full)
TILE#_HLC_fifo_underflow_signal	When high, this signal indicates a FIFO underflow has occurred (pulling when the FIFO is empty)
TILE#_HLC_fifo_write_pointer [1:0]	Current value of the write pointer for the push FIFO (zero-indexed)
TILE#_HLC_fifo_read_pointer [1:0]	Current value of the read pointer for the pull FIFO (zero-indexed)
TILE#_HLC_push_fifo(Y) [31:0]	Represents the current values in the push FIFO
TILE#_HLC_pull_fifo(Y) [31:0]	Represents the current values in the pull FIFO
TILE#_HLC_program_current_instruction [11:0]	Signal which shows the opcode of the current instruction being executed by the HLC
TILE#_HLC_register(Y) [31:0]	Represents the current values of the HLC registers R0 through R3
TILE#_HLC_program_interrupt_number [31:0]	Signal which indicates which interrupt has been triggered by the HLC; the default values is 0xFFFF, otherwise the value represents the interrupt value from the last 6 bits of the interrupt opcode
TILE#_HLC_program_interrupt_flag	This signal is high when the current instruction being executed by the HLC is an interrupt
TILE#_HLC_hlc_counterX_match2_load_en	Enable which goes high when the HLC is loading the corresponding counter match reference 2 value using the MOV_T2 instruction; this matches the appropriate TILE#_COUNTER_X_hlc_match2_load_en signal
TILE#_HLC_hlc_counterX_match1_load_en	Enable which goes high when the HLC is loading the the corresponding counter match reference 1 value using the MOV_T1 instruction; this matches the appropriate TILE#_COUNTER_X_hlc_match1_load_en signal

Copyright © 2023 Texas Instruments Incorporated

表 3-8. 高级控制器块跟踪信号（续）

跟踪信号名称	说明
TILE#_HLC_hlc_counterX_load_en	HLC 使用 MOV 指令装载对应计数器值时，该信号置位；它与相应的 TILE#_COUNTER_X_hlc_counter_load_en 信号一致。
TILE#_HLC_counter_hlc_load_value [31:0]	根据 HLC 执行的指令，装载匹配参考值 1、匹配参考值 2 或计数器值；该值与相应的 TILE#_COUNTER_X_hlc_counter_load_val 信号相对应。
TILE#_HLC_spi_export_enable	该信号表示是否已启用通过 SPI RX 缓冲区导出数据。
TILE#_HLC_reconfig_pipeline_enable	该信号表示是否已启用可重配置流水线模式；此模式会同时影响计数器和 HLC 子模块。
TILE#_HLC_alternate_event_clb_async_output(N)	这组信号表示来自 AOC 子模块的 CLB 异步输出，是 HLC 使用的备用事件之一。
TILE#_HLC_alternate_event_clb_output(N)	这组信号表示来自 OUTLUT 子模块的 CLB 输出，是 HLC 使用的备用事件之一。
TILE#_HLC_hlc_event_trigger(31..0)	这组信号表示 HLC 事件的触发源。有关各触发信号与 HLC 事件触发值的对应关系，请参阅器件技术参考手册。备用事件不属于此信号集。
TILE#_HLC_alternate_event_input_selectY	该选择项表示相应 HLC 事件是否使用备用事件集。
TILE#_HLC_spi_shift_value [4:0]	此值用于选择从 32 位 R0 寄存器导出到 SPI RX 缓冲区的 16 位数据。例如，值为 1 时选择 R0 寄存器的位 16:1。
TILE#_HLC_spi_event_trigger [4:0]	此信号用于选择触发数据导出到 SPI RX 缓冲区的 HLC 事件。可选事件来自静态开关块，详见器件技术参考手册。
TILE#_HLC_programY_event_source [31:0]	该值表示相应 HLC 事件使用的触发源。例如，当 TILE#_HLC_program0_event_source 的值为 1 且备用事件输入选择设为 0 时，事件 0 的源为计数器 0 的匹配参考值 2；该事件会触发 HLC program 0。
TILE#_HLC_counterX_value [31:0]	当前对应计数器的值；它表示执行 HLC 指令时，作为操作数的 C0、C1 或 C2 所使用的值。
TILE#_HLC_program_global_load_en	CLB 使能信号
TILE#_HLC_program_reset	CLB 复位信号
TILE#_HLC_program_clock	CLB 时钟信号

表 3-9. 查找表块跟踪信号

跟踪信号名称	说明
TILE#_LUT_X_output	查找表子模块的输出信号
TILE#_LUT_X_output_equation [15:0]	表示查找表逻辑方程的值
TILE#_LUT_X_inputY	查找表子模块的输入信号

表 3-10. 输出查找表块跟踪信号

跟踪信号名称	说明
TILE#_OUTLUT_N_output	输出查找表子模块的输出信号
TILE#_OUTLUT_N_output_equation [7:0]	表示输出查找表逻辑方程的值
TILE#_OUTLUT_N_inputX	输出查找表子模块的输入信号

Copyright © 2023 Texas Instruments Incorporated

4 示例

C2000Ware 随附了 CLB 工具示例，展示如何配置 CLB 以实现各种应用场景。所有带 CLB 的器件均提供示例，但由于较新 CLB 类型新增了一些旧版本不具备的功能（例如异步输出整形块），并非所有示例都适用于所有器件。以下大多数示例针对 F28003x 器件进行说明，例外是示例 15，该示例适用于 F2838x 器件。

4.1 基础示例

这些示例旨在展示每个 CLB Tile Design 中子模块的基本功能。每个示例介绍若干子模块，以及如何组合使用这些子模块实现简单逻辑。第 2.1 节列出的链接中还提供了更多示例。

若要更好地了解 Tile Design 的配置及子模块之间的连接关系，请按照以下步骤生成 CLB 方框图（步骤见第 3.4 节）。

4.1.1 CLB 空白项目

该示例是一个空白 CLB 项目，包含构建后步骤，可生成“.OUT”目标二进制文件、仿真“.VCD”文件以及 HTML 方框图。

4.1.2 示例 3 – PWM 生成

此示例将 CLB 逻辑单元配置为辅助 PWM 生成器，并使用组合逻辑（LUT）、状态机（FSM）、计数器和高级控制器（HLC），演示如何利用 CLB 生成 PWM 输出。

PWM 生成器以 CLBCLK 频率运行。FSM 用于置位或清除 PWM。CMP 匹配事件会置位 PWM，该事件与 COUNTER_0 的 match2 相连。零值匹配事件（Z）会清除 PWM，该事件与 COUNTER_0 的 match1 输出相连。

PWM 寄存器配置为使用活动寄存器和影子寄存器，此配置通过 HLC 块完成。HLC 用于在周期匹配事件 match1 发生时生成中断。中断发生后，新的计数器匹配值会加载到 HLC 寄存器（R0）中，随后从该寄存器写入 COUNTER_0 的 match2 寄存器。这样会更新 CMP 匹配值，进而改变正向占空比。本示例在两个正向占空比值之间交替切换。图 4-1 展示了 PWM 生成器的基本工作方式。请注意，下一个周期的占空比会发生变化。

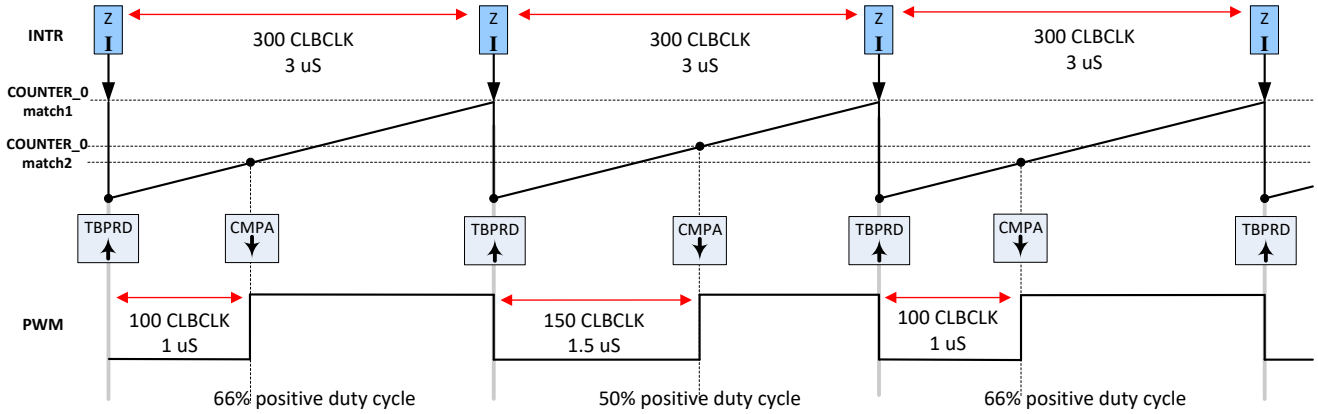


图 4-1. 示例 3：生成的 PWM 波形

Copyright © 2023 Texas Instruments Incorporated

CLB 逻辑单元接收 PWM 使能信号作为输入，并向 CPU 生成中断。CLB 逻辑单元通过计数器计数，直到达到设定的周期和比较事件值。当计数器达到比较事件匹配值（输出 match2）时，输出置为高电平并保持，直到计数器达到周期匹配值（输出 match1）或触发计数器复位。发生周期事件或复位后，计数器复位为 0，输出变为低电平，计数器随即重新开始递增。此输出逻辑通过在 FSM 中输入逻辑方程进行配置。本示例的周期为 300 个 CLBCLK 周期（3 µs）。比较事件发生在 100 个 CLBCLK 周期（1 µs）或 150 个 CLBCLK 周期（1.5 µs）时。

PWM 信号可通过将 FSM 输出馈入 OUTLUT_4 来查看。若要在示波器 上查看该输出，必须通过 Output X-BAR 将其传送到 GPIO Mux。

按照以下步骤运行此示例：

1. 在 CCS v9.0 或更高版本中，单击“Project → Import CCS Projects …”。2. 转到 CLB 工具示例目录。路径为：a. [C2000Ware]\driverlib\f2837xd\examples\cpu1\clb\ccs，或 b. [C2000Ware]\driverlib\f28004x\examples\clb\ccs，或 c. [C2000Ware]\driverlib\f2838x\examples\c28x\clb\ccs

以下说明均假定使用上述 C2000Ware 目录。
3. 选择项目“clb_ex3_auxiliary_pwm”，然后单击“Finish”（完成）。
4. 在 CCS Project Explorer（项目资源管理器）窗口中展开项目，并打开“clb_ex3_aux_pwm.syscfg”文件。
5. 检查 tile 配置，并查看 LUT4_0、COUNTER_0、FSM_0 中的逻辑表达式，以及 HLC 和输出 LUT 的配置。6. 在 CCS 菜单中选择“Project → Build Project”（构建项目）。
7. 打开“Debug/syscfg/clb.html”文件，查看 CLB Tile 方框图。
8. [可选] 有关如何运行 CLB 仿真的说明，请参阅第 3.5.3 节。
9. 若要查看 PWM 和中断信号，请连接示波器，并在程序运行时监测以下引脚。下表列出了各开发板对应的监测引脚。

Signal	F28379D LaunchPad	F280049 controlCARD	F28388D controlCARD
Interrupt	GPIO0 on pins J4/40	pin 49 (GPIO0)	pin 49 (GPIO0)
Auxiliary PWM	OutputXBAR1 signal on pins J4/34	pin 53 (OutputXBAR1)	pin 53 (OutputXBAR1)

10. 打开 CCS Expressions（表达式）窗口，并添加程序变量 *dutyValue*。程序运行期间，每次处理 CLB 中断时，CMPA 值都会在 100 和 150 个 CLBCLK 周期之间交替。信号波形应如上方时序图所示。请注意，PWM 周期保持不变，但正向占空比会在 50% 和 66% 之间交替。可以在中断服务例程中修改 *dutyValue* 变量。

4.1.3 示例 7 – 状态机

*Designing With The C2000 Configurable Logic Block*本例将逐步介绍如何使用 CLB 设计应用，并按照设计流程逐项展开。该示例使用一个 CLB 逻辑单元（tile）中的全部子模块来实现完整系统。

4.1.4 示例 13 – 推挽接口

本示例展示了 PUSH-PULL 接口的用法。示例使用多个 COUNTER 子模块、HLC 子模块、FSM 子模块和 OUTLUT 子模块，并通过 PUSH-PULL 接口配合 GP 寄存器更新 COUNTER 子模块的事件频率。

4.1.5 示例 14 – 多 CLB 逻辑单元（tile）设计

本示例将一个 CLB 逻辑单元（tile）的输出传递到另一个 CLB 逻辑单元（tile）的输入，再将第二个 CLB 逻辑单元的输出导出到 GPIO，展示如何串联两个 CLB 逻辑单元（tile）。

Copyright © 2023 Texas Instruments Incorporated

4.1.6 示例 15 – Tile 间延迟

本示例将 GPIO 输出经 INPUT XBAR 和 CLB XBAR 输入 CLB 逻辑单元（tile），再由该逻辑单元将信号转发到下一个 CLB 逻辑单元（tile）。本次信号仅经过 CLB XBAR，不经过 INPUT XBAR。这样可以说明信号在 CLB 逻辑单元（tile）之间传递时会产生延迟，而且该延迟未经表征。用户应避免在 CLB 逻辑单元（tile）之间传递有时序要求的信号。CLB 内的 COUNTER 模块会按周期计数，用于测量延迟。

4.1.7 示例 16 – 粘合逻辑

本示例将逐步讲解如何把 FPGA/CPLD 中的自定义逻辑迁移到 C2000™ 微控制器。

4.1.8 示例 18 – AOC

本示例使用异步输出整形块，对输入到 CLB 的信号执行异步 AND 运算。该模块仅适用于 CLB Type 2 及更高类型。

4.1.9 示例 19 – AOC 释放控制

本示例使用异步输出整形块，对输入到 CLB 的信号进行异步置位和释放。该模块仅适用于 CLB Type 2 及更高类型。

4.1.10 示例 20 – CLB XBAR

本示例使用 CLB INPUTXBAR 和 CLB OUTPUTXBAR，将 GPIO 输入信号送入 CLB 逻辑单元（tile），并将 CLB 逻辑单元（tile）的输出信号送至 GPIO。这些 XBAR 是否可用取决于器件。

4.2 入门示例

这些示例的复杂度稍高，可用于构建更实用的 CLB 外设示例和实现。第 2.1 节列出的链接中还提供了更多示例。

若要更好地了解 Tile Design 的配置及子模块之间的连接关系，请生成 一幅 CLB 方框图；具体步骤见第 3.4 节。

4.2.1 示例 1 – 组合逻辑

本示例旨在避免 PWM 对的两个输出同时处于高电平或低电平。PWM 模块 1 和 2 配置为基于固定频率向上计数模式生成相同波形。PWM 模块 2 的时基与 PWM 模块 1 同步，如图 4-2 所示。

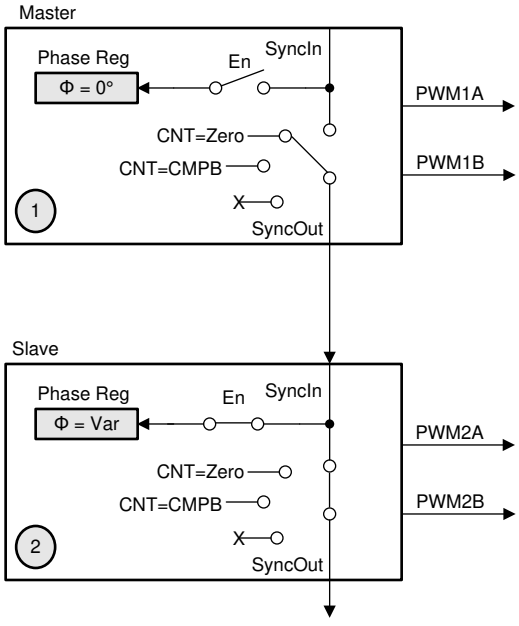


图 4-2. 示例 1：EPWM 同步

Copyright © 2023 Texas Instruments Incorporated

PWM 波形会在不同时间点将每个模块的两个输出强制为相同电平：
有时同时为高电平，有时同时为低电平，如图 4-3 所示。

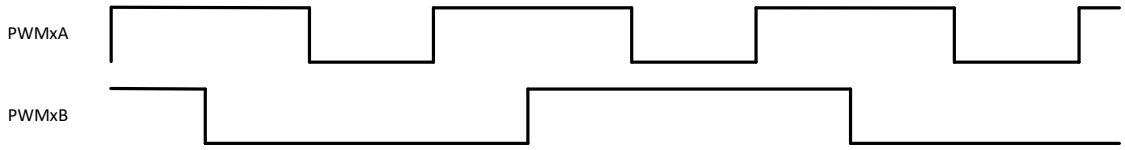


图 4-3. 示例 1：PWM 测试模式

CLB 会调整这些波形，以消除两个输出同时为高电平或同时为低电平的情况。这是一个简单的组合逻辑示例。逻辑有三种工作模式：正常、高有效和低有效。在正常模式下，PWM 信号不经修改，直接通过 CLB。在高有效模式下，逻辑会防止 PWM 引脚同时输出逻辑“1”。类似地，在低有效模式下，两个 PWM 引脚不得同时输出逻辑“0”。例如，处于低有效模式且两个 PWM 信号均为低电平时，两个 PWM 的输出都会被强制为高电平。更多详情请参阅表 4-2。模式由一个 2 位字段选择，如表 4-1 所示。

表 4-1. 示例 1：工作模式

Mode Name	Type	[MODE 1]	[MODE 0]
M0	Normal	0	0
M1	Active Low	0	1
M2	Active High	1	0
M3	Reserved	1	1

图 4-4 所示的逻辑电路实现了上述模式。输出信号名称后缀“_m”表示该信号可能已由 CLB 修改。

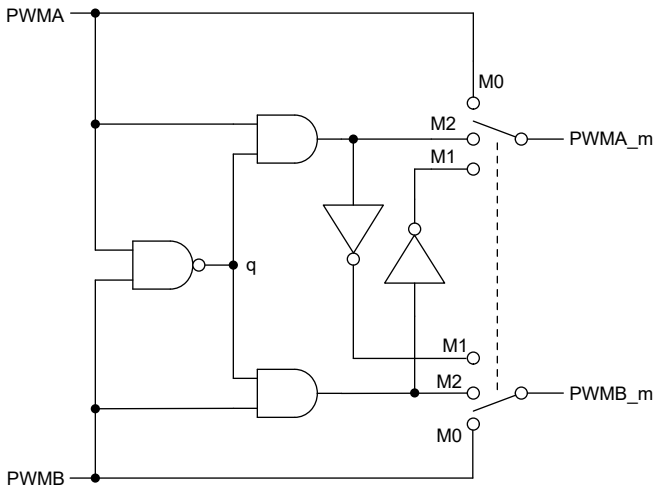


图 4-4. 示例 1：逻辑图

上述逻辑可用两个 4 输入 LUT 实现，每个输出信号各用一个。因此只需使用一个 CLB 逻辑单元（tile）的一小部分。本示例仅由 CLB 修改 PWM1 模块的信号；PWM2 的信号则直接连接到器件引脚，以供比较。图 4-5 展示了 PWM1 的输入和输出波形（模式 1 和 2）。图中用绿色标出 CLB 逻辑强制输出为高电平或低电平的区域。

Copyright © 2023 Texas Instruments Incorporated

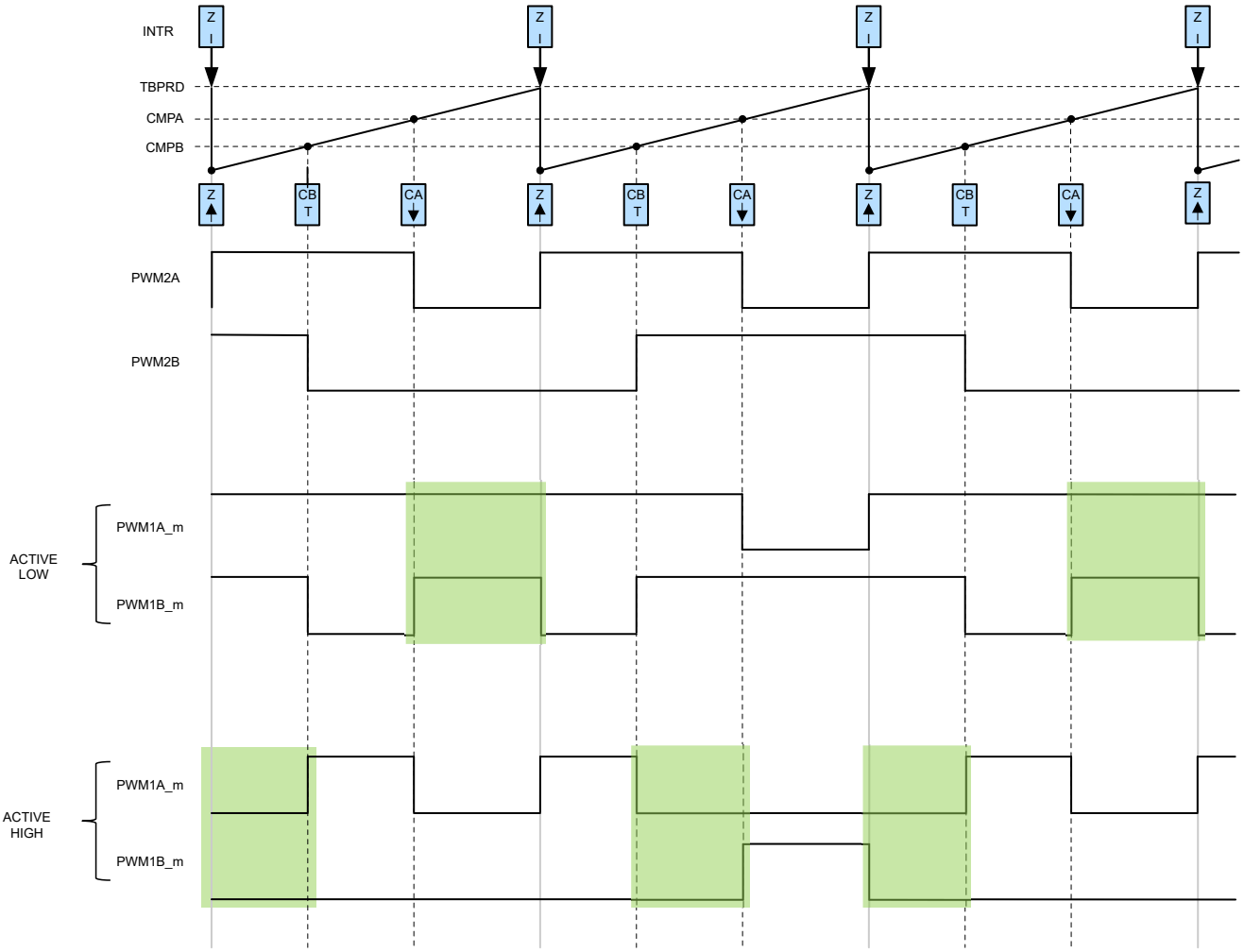


图 4-5. 示例 1：生成的 PWM

表 4-2. PWM 输出

Operating Mode	Original PWM A Output	Original PWM B Output	PWM A Output	PWM B Output
Normal	0	0	0	0
	0	1	0	1
	1	0	1	0
	1	1	1	1
Active Low	0	0	1	1
	0	1	0	1
	1	0	1	0
	1	1	1	1
Active High	0	0	0	0
	0	1	0	1
	1	0	1	0
	1	1	0	0

Copyright © 2023 Texas Instruments Incorporated

所需逻辑由 4 输入 LUT 0 和 LUT 1 实现。每个 LUT 均连接到两个 PWM 信号和软件“mode”变量的两个最低有效位；这些位写入 GPREG 寄存器。CLB 输出连接到 PWM1A 和 PWM1B 信号，这些信号随后分别连接到 GPIO 引脚和 1。图 4-6 示意了与 CLB 逻辑单元的连接关系。

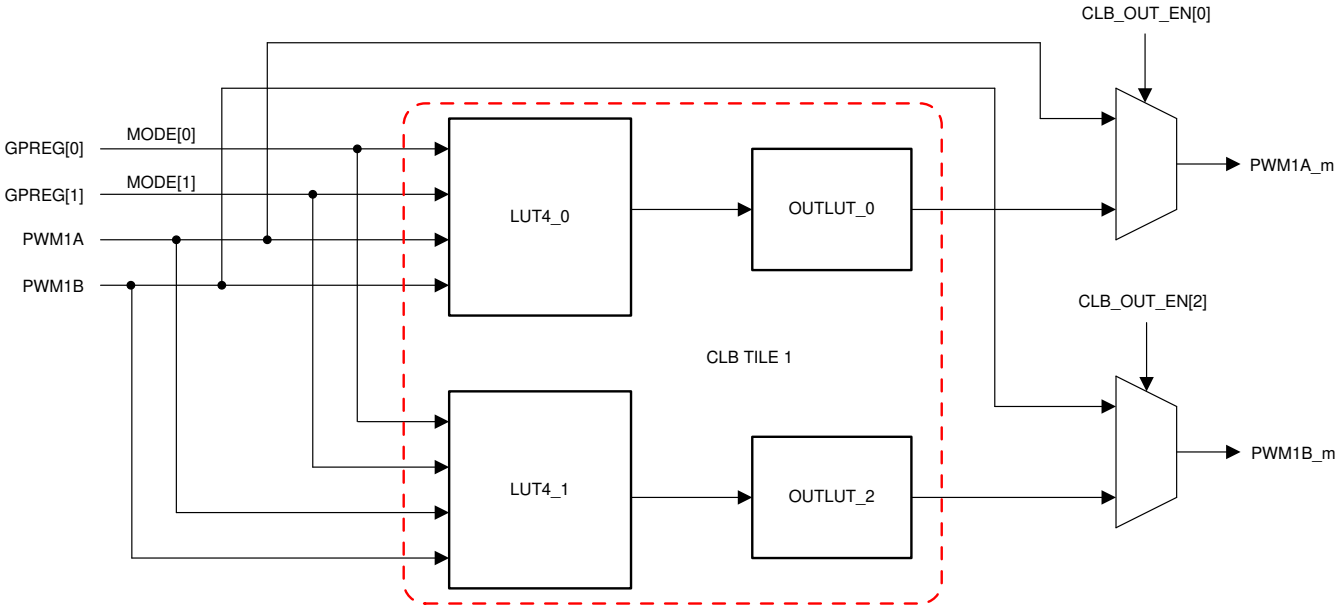


图 4-6. 示例 1：CLB 配置

按照以下步骤运行此示例：

1. 单击“Project → Import CCS Projects…”。2. 转到 CLB 工具示例目录。路径为：a. [C2000Ware]\driverlib\f2837xd\examples\cpu1\clb\ccs，或 b. [C2000Ware]\driverlib\f28004x\examples\clb\ccs，或 c. [C2000Ware]\driverlib\f2838x\examples\c28x\clb\ccs。以下说明均假定使用上述 C2000Ware 目录。3. 选择项目“clb_ex1_combinatorial_logic”，然后单击“Finish”（完成）。4. 在 CCS Project Explorer（项目资源管理器）窗口中展开“clb_ex1_combinatorial_logic”项目，并打开“clb_ex1_combinatorial_logic.syscfg”文件。5. 检查 tile 配置，并查看 LUT4_0 和 LUT4_1 中的逻辑表达式以及输出 LUT 的配置。6. 在 CCS 菜单中选择“Project → Build Project”（构建项目）。7. 监测相应引脚。a. 文档列出了 F28379D 的 Launchpad 和 F28388D Experimenter 套件上应监测 PWM 的引脚，但未列出其他器件的 Launchpad 引脚。有关可用引脚及其配置的详细信息，请参阅器件数据手册。8. 打开 CCS Expressions（表达式）窗口。9. [Optional] 有关如何运行 CLB 仿真的说明，请参阅第 3.5.3 节。

如果程序运行在 F28379D LaunchPad 开发板上，可分别通过 J4/40 和 J4/39 引脚监测 PWM1A 和 PWM1B 信号。程序运行时，请使用示波器监测这些引脚上的信号。

如果程序运行在安装了 F28388D ControlCARD 控制卡的 Experimenter 套件上，可在以下引脚上找到这些信号：引脚 49 和 51。

打开 CCS Expressions（表达式）窗口，并添加程序变量“mode”。当 mode 为默认值 0 时，PWM 信号未经修改，直接通过 CLB。停止程序，将 mode 改为 1，然后重新启动程序。信号波形应如上方时序图所示。重复此操作，将 mode 改为 2，并确认信号与前一时序图所示一致。

Copyright © 2023 Texas Instruments Incorporated

4.2.2 示例 2 – GPIO 输入滤波器

本示例展示如何使用有限状态机（FSM）和计数器实现简单的毛刺滤波器，例如用它滤除 GPIO 输入信号中不需要的短脉冲。

图 4-7 示意了毛刺滤波器的工作方式。输入数字信号按 CLB 时钟速率采样，计数器统计输入连续处于高电平或低电平的采样次数。如果该次数达到或超过指定的采样窗口长度，滤波器输出就会变为与输入相同的电平；否则，滤波器输出保持不变。图 4-7 示意了该滤波器的工作方式。

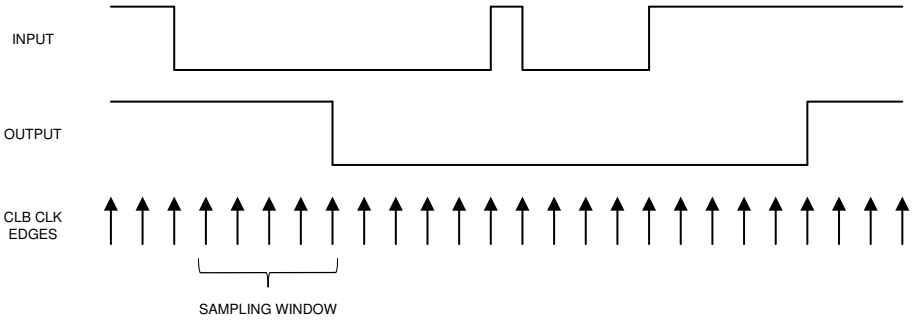


图 4-7. 示例 2：GPIO 毛刺滤波示例

CLB 配置使用一个 LUT4 反转输入信号，并使用两个计数器统计脉冲数：一个统计高电平脉冲，另一个统计低电平脉冲。当任一计数器达到采样窗口长度时，其 match1 输出上会出现一个脉冲。本示例将滤波器的采样窗口长度设为八。FSM 锁存该脉冲，并通过简单的逻辑方程确定 S0 状态输出所需的电平。一个输出 LUT 用于将 FWM 输出传送到外设信号多路复用器，再连接到 GPIO0。CLB 配置如图 4-8 所示。

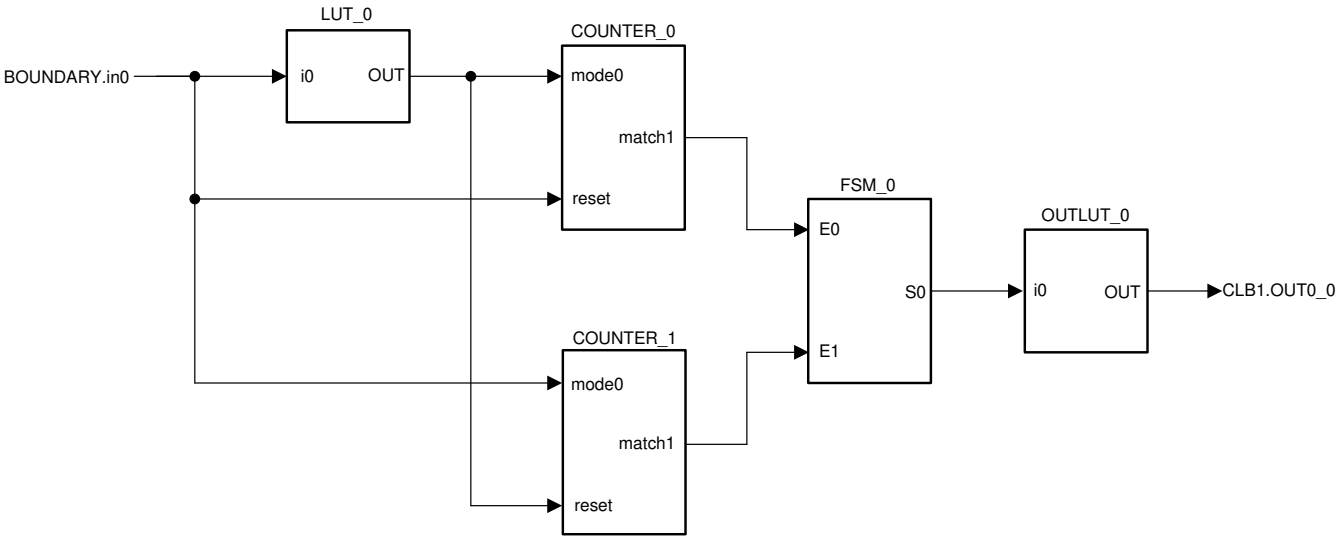


图 4-8. 示例 2：CLB 配置

示例代码将 ePWM1 模块配置为生成测试激励。

按照以下步骤运行此示例：

1. 在 CCS v9.0 或更高版本中，单击“Project → Import CCS Projects …”。2. 转到 CLB 工具示例目录。路径为：a. [C2000Ware]\driverlib\f2837xd\examples\cpu1\clb\ccs，或 b. [C2000Ware]\driverlib\f28004x\examples\clb\ccs，或 c. [C2000Ware]\driverlib\f2838x\examples\c28x\clb\ccs

以下说明均假定使用上述 C2000Ware 目录。

Copyright © 2023 Texas Instruments Incorporated

3. 选择“glitch_filter”项目，然后单击“Finish”（完成）。4. 在 CCS Project Explorer（项目资源管理器）窗口中展开“glitch_filter”项目，并打开“tile.syscfg”文件。5. 检查 tile 配置，并查看 LUT4_0、COUNTER_0、COUNTER_1 和 FSM_0 子模块的设置。确认这些设置与上述示例说明一致。6. 在 CCS 菜单中选择“Project → Build Project”（构建项目）。7. [Optional] 有关如何运行 CLB 仿真的说明，请参阅第 3.5.3 节。

如果程序运行在 F28379D LaunchPad 开发板上，可分别通过 J4/40 和 J4/39 引脚监测 PWM1A 和 PWM1B 信号。如果程序运行在安装了 F280049 或 F28388D ControlCARD 控制卡的 Experimenter 套件上，则可分别通过引脚 49 和 51 监测信号。

打开 CCS Expressions（表达式）窗口并添加程序变量“cglitch”。运行程序，同时观察 PWM1A 和 PWM1B 信号。暂停程序，更改“cglitch”的值，然后重新启动程序（将表达式窗口设为“continuous refresh”（连续刷新）会更方便）。当取值为 7 或更小时，由于毛刺宽度小于采样窗口，滤波器应能滤除毛刺。当“cglitch”大于 7 时，两个输出上都应出现毛刺。还请注意，PWM1A 的边沿比 PWM1B 略有延迟，这是所用滤波方法造成的。

图 4-9 显示了毛刺宽度低于和高于采样窗口设定值 8 时输出引脚上预期的波形。

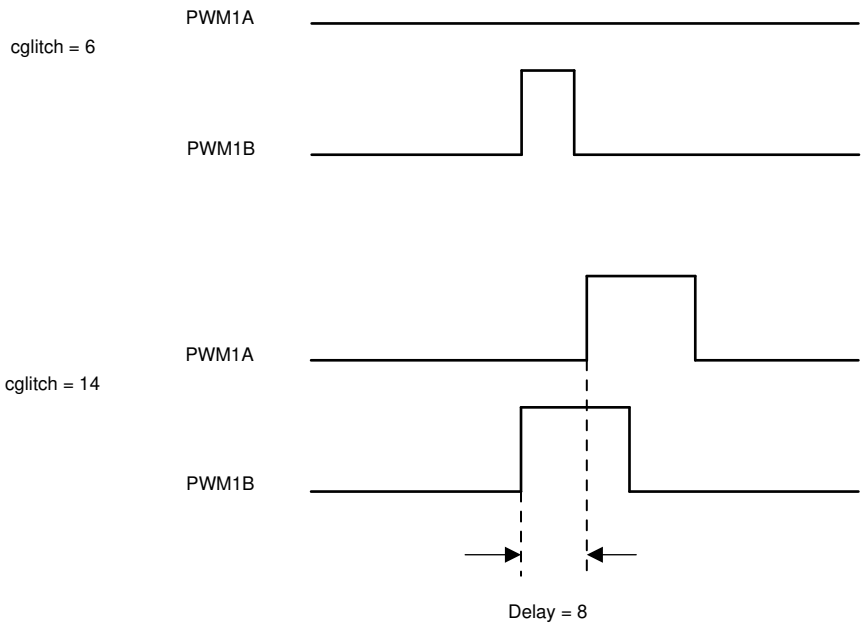


图 4-9. 示例 2：GPIO 毛刺宽度

4.2.3 示例 4 – PWM 保护

本示例扩展了示例 1 的功能，以确保高电平有效的互补 PWM 对始终具有最小死区时间，不受 PWM 生成模块配置的影响。示例通过配置四个独立的 CLB 逻辑单元（tile），为四个 PWM 模块实现保护。PWM 模块 1 至 4 的输出分别由 CLB 逻辑单元（tile）1 至 4 处理。

保护功能由程序变量“mode”启用。当该变量设为 0（默认值）时，PWM 信号未经修改，直接传至输出引脚。设为 1 时，CLB 会调整 PWM 输出以确保存在死区时间。

Copyright © 2023 Texas Instruments Incorporated

按照以下步骤运行此示例：

1. 在 CCS v9.0 或更高版本中，单击“Project → Import CCS Projects…”
2. 转到 CLB 工具示例目录。路径为：

   a. [C2000Ware]\driverlib\f2837xd\examples\cpu1\clb\ccs，或 b. [C2000Ware]\driverlib\f28004x\examples\clb\ccs，或 c. [C2000Ware]\driverlib\f2838x\examples\c28x\clb\ccs

   以下说明均假定使用上述 C2000Ware 目录。

3. 选择“clb_ex4_pwm_protection”项目，然后单击“Finish”（完成）。4. 在 CCS Project Explorer（项目资源管理器）窗口中展开“clb_ex4_pwm_protection”项目，并打开“clb_ex_pwm_protection.syscfg”文件。5. 在 CCS 菜单中选择“Project → Build Project”（构建项目）。6. 使用示波器观察每块 LaunchPad/Docking Station 开发板上以下引脚处的 PWM 信号对。

表 4-3. 示例 4：信号连接

PWM	Tile	F28379D LaunchPad	F280049 LaunchPad	F28388 Dock Station
1A	1	J4/40	J8/60	49
1B	1	J4/39	J8/59	51
2A	2	J4/38	J8/56	53
2B	2	J4/37	J8/55	55
3A	3	J4/36	J4/36	50
3B	3	J4/35	J4/35	52
4A	4	J8/80	J8/58	54
4B	4	J8/79	J8/57	56

7. 打开 CCS Expressions（表达式）窗口，并添加程序变量“mode”。
8. 运行程序，确认每对 PWM 信号之间均不存在死区时间。
9. 暂停程序，将 mode 改为 1，然后重新运行。此时应能观察到每对 PWM 信号之间的上升沿死区时间。死区时间由加载到两个 CLB 计数器中的 match_1 值设定；本示例将该值任意设为 10。

4.2.4 示例 5 – 事件窗口

本示例使用 CLB 的计数器、FSM 和 HLC 子模块实现事件计时功能，用于检测中断服务程序是否响应过慢。示例配置四个 PWM 模块以向上计数模式运行，并在定时器零值匹配事件时生成低到高的边沿。该零值匹配事件还会触发 PWM ISR；在本示例中，ISR 中包含长度可变的占位负载。ISR 结束时会写入 CLB GP 寄存器，以指示 ISR 已执行完毕。

CLB 模块会检测 PWM 定时器的零值事件，并由此启动计时器。计时器的“match 2”计数值设为对应 PWM ISR 的最大预期执行时长。如果在计数达到 match 2 之前未写入 GP 寄存器，HLC 就会触发 CLB 中断。四个 PWM 模块和 CLB 逻辑单元均以相同方式配置。

Copyright © 2023 Texas Instruments Incorporated

图 4-10 概述了一个 tile 的工作方式。上半部分显示 PWM 模块的配置，该配置生成固定频率波形：每次计数器零值匹配时产生上升沿，并在
比较 A 匹配时产生下降沿。零值匹配事件会生成 CPU 中断；本例旨在当 PWM ISR 未在指定时间内完成时触发 CLB 中断。

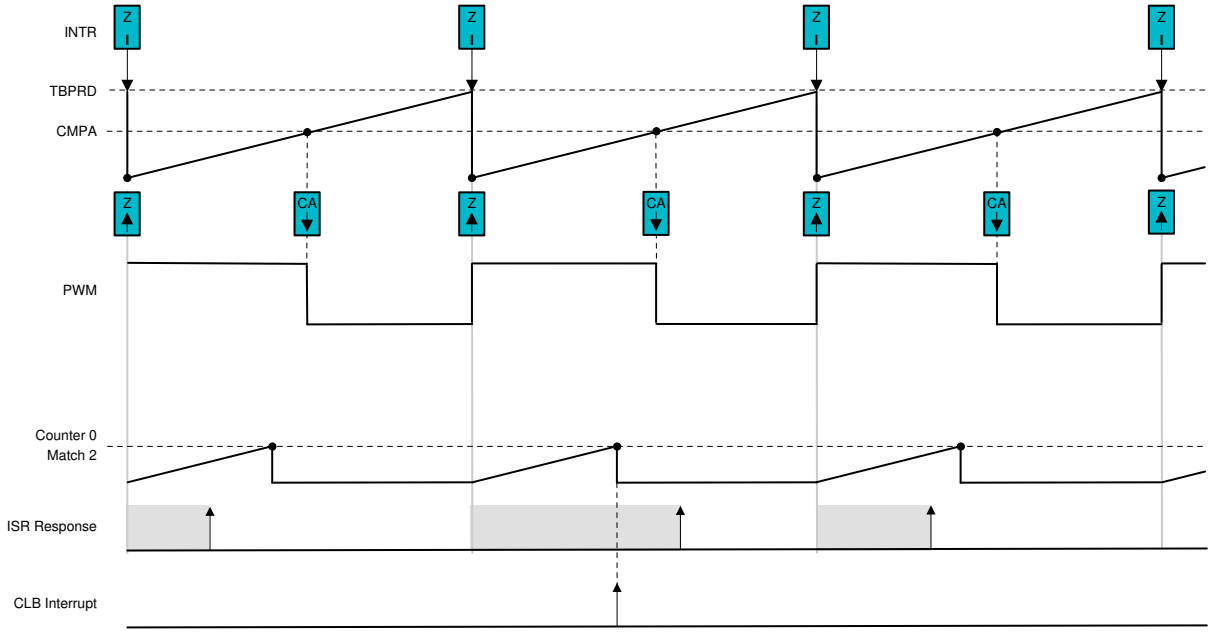


图 4-10. 示例 5：事件窗口配置

下半部分显示 CLB 计数器，该计数器会在 PWM ISR 开始时启动计数。如果 ISR 在达到 Match 2 值之前未响应，就会生成中断。CLB ISR 包含“ESTOP”指令，其作用类似于程序中的软件断点。

按照以下步骤运行此示例：

1. 在 CCS v9.0 或更高版本中，单击“Project → Import CCS Projects …”。2. 转到 CLB 工具示例目录。路径为：a. [C2000Ware]\driverlib\f2837xd\examples\cpu1\clb\ccs，或 b. [C2000Ware]\driverlib\f28004x\examples\clb\ccs，或 c. [C2000Ware]\driverlib\f2838x\examples\c28x\clb\ccs

以下说明均假定使用上述 C2000Ware 目录。
3. 选择“clb_ex5_event_window”项目，然后单击“Finish”（完成）。4. 在 CCS Project Explorer（项目资源管理器）窗口中展开“clb_ex5_event_window”项目，并打开“clb_ex5_event_window.syscfg”文件。5. 在 CCS 菜单中选择“Project → Build Project”（构建项目）。

打开 CCS Expressions（表达式）窗口，并添加四个程序变量：“payload_x”，其中“x”的取值为 1 至 4。观察程序启动时，所有 payload 变量均已设为 45。该负载通过每个 PWM ISR 中的“for”循环实现；每次迭代需要 12 个周期，因此负载值 45 对应约 540 个周期。

打开 .syscfg 文件，检查四个 CLB 模块中 counter 0 的 match 2 设置。请注意，所有计时器限值均设为 3200。

使用默认负载运行程序，并确认不会触发 CLB 中断。然后停止程序并增大任一负载值。重新运行程序，检查是否有 ISR 超出时限。请注意，由于各 PWM 未同步，最坏情况下的 ISR 延迟等于所有负载时间与中断开销之和。

Copyright © 2023 Texas Instruments Incorporated

4.2.5 示例 6 – 信号生成与检查

本示例使用 CLB1 生成矩形波，并使用 CLB2 检查 CLB1 生成的矩形波是否超出设定的占空比和周期限制。

CLB1：本示例使用 CLB 的 counter 和 FSM 子模块实现矩形脉冲生成器。counter0 会在用户设定的 Match1 和 Match2 值处生成事件。Match2 值决定所生成波形的周期，而（Match2 – Match1）值决定导通时间（ON time）。FSM 使用计数器生成的这些事件来产生波形：在 Match1 事件时置位输出，在 Match2 事件时清除输出。因此，状态位 S0 表示生成的输出波形。该输出随后由 CLB1 的 Output 4 引出，再经 CLB X-Bar 传送至 CLB2。In0 用作软件控制的波形生成使能信号，并通过 CLB1 Output 5 传送至 CLB2。

CLB2：本示例使用 CLB 的 LUT、counter、FSM 和 HLC 子模块实现一个检查器，用于检查 CLB1 生成的输出。CLB2 的信号连接关系如下：

CLB1 Output 4 → CLB X-Bar AUXSIG0 → CLB2 in1 (via Global Mux)

CLB1 Output 5 → CLB X-Bar AUXSIG1 → CLB2 in2 (via Global Mux)

counter0 在 In1 所接收信号的导通期间计数。counter0 的 Match1 值设为占空比限值。若发生 match1 事件，即表示导通时间已超过目标值。

counter1 在 In1 接收信号的上升沿复位并开始计数。Counter1 的 Match1 值设为周期限值。若发生 match1 事件，即表示周期已超过目标值。

状态机（FSM1 S0）用于检测 In1 上接收信号的上升沿，并将其用作 counter 1 的复位信号。

上述任一计数器 match1 事件发生时，HLC 都会向 CPU 生成中断，以指示发生错误。

Copyright © 2023 Texas Instruments Incorporated

图 4-11 概述了各 tile 的工作方式。match1 事件会生成 CPU 中断；本例旨在当 CLB2 内检测到错误时触发 CLB 中断。

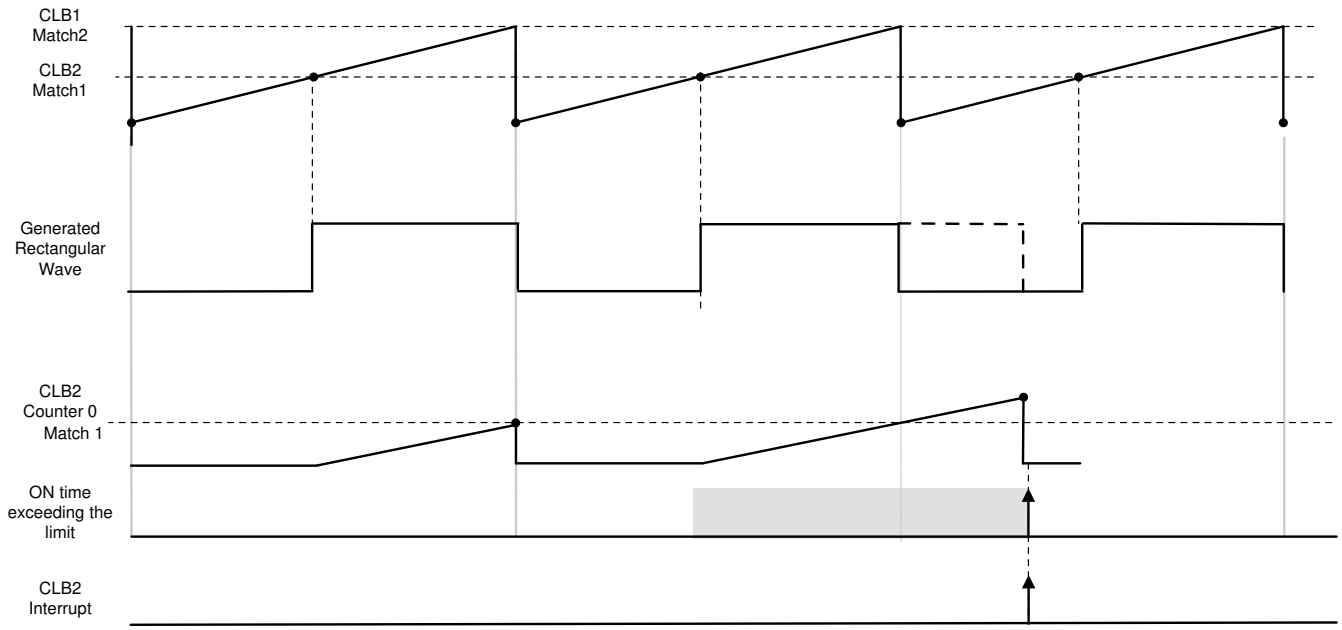


图 4-11. 示例 6：占空比超过预设值

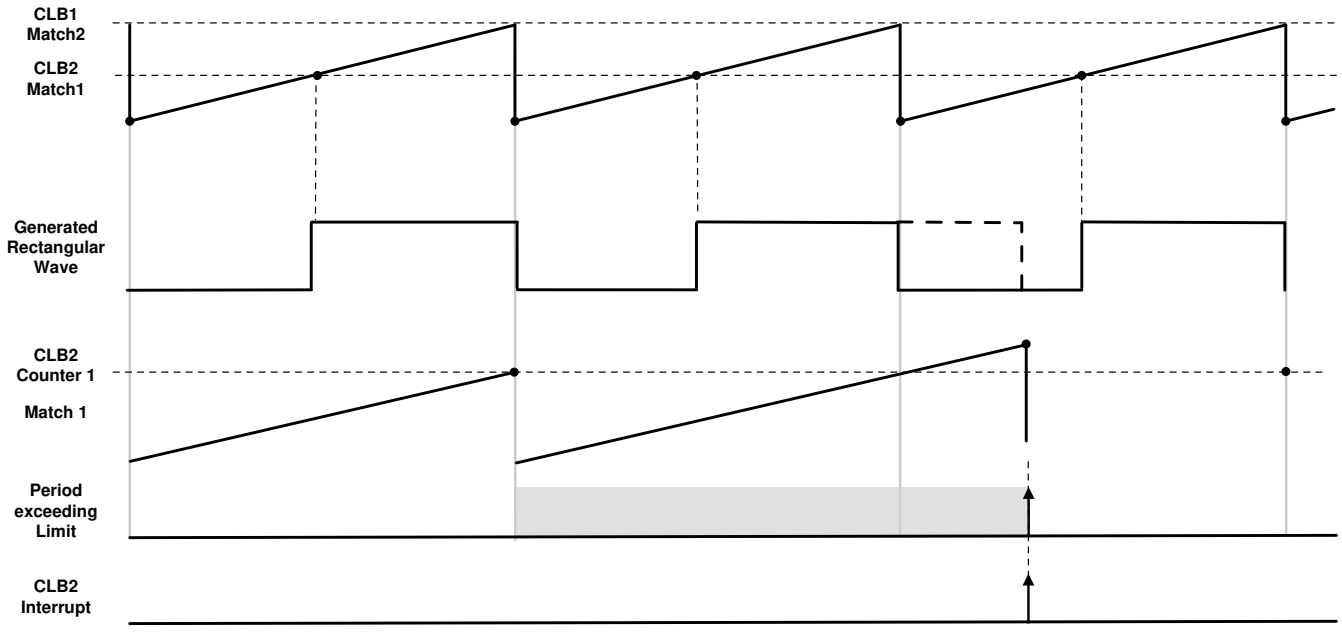


图 4-12. 示例 6：周期超过预设值

Copyright © 2023 Texas Instruments Incorporated

下半部分显示 CLB 计数器，该计数器会在导通时间开始时启动。第一幅图表示占空比检查，第二幅表示周期检查。两种情况下，达到 Match 1 值都会触发中断。CLB ISR 中的“ESTOP”指令会使程序暂停，作用类似于软件断点。

按照以下步骤运行此示例：

1. 在 CCS v9.0 或更高版本中，单击“Project → Import CCS Projects …”。2. 转到 CLB 工具示例目录。路径为：a. [C2000Ware]\driverlib\f2837xd\examples\cpu1\clb\ccs，或 b. [C2000Ware]\driverlib\f28004x\examples\clb\ccs，或 c. [C2000Ware]\driverlib\f2838x\examples\c28x\clb\ccs

以下说明假定使用上述 C2000Ware 目录。3. 13. 选择“clb_ex6_siggen”项目，然后单击“Finish”（完成）。4. 14. 在 CCS Project Explorer（项目资源管理器）窗口中展开“clb_ex6_siggen”项目，并打开“clb_ex6_siggen.syscfg”文件。5. 在 CCS 菜单中选择“Project → Build Project”（构建项目）。

在 CCS 窗口中打开 SysCfg 文件（clb_ex6_siggen.syscfg），检查 counter 0 模块的 match 1/2 设置。更改这些值即可调整生成输出的占空比和周期。

检查 CLB2 模块中 counter 0/1 的 match 1 设置。更改这些值即可调整对生成输出进行检查的占空比和周期限值。

使用默认值运行程序，并确认不会触发 CLB 中断。然后更改这些值以产生错误（例如，将 CLB2 Counter1 Match1 改为 400）。重新构建并运行程序，确认代码会在 CLB2 中断服务程序中暂停。

4.2.6 示例 8 – 外部与门

本示例将来自两个 GPIO 的两个外部信号经 Input X-BAR 和 CLB X-BAR 送入 CLB 逻辑单元（tile）。这两个信号在 CLB 模块内部执行 AND 运算。随后使用 Output X-BAR 将与门输出连接到 GPIO。

4.2.7 示例 9 – 定时器

本示例使用 COUNTER 模块生成定时事件，并展示 GP Register 的用法。通过设置或清除 GP 寄存器中的位，可启动、停止或改变定时器的计数方向。定时事件输出（1 个时钟周期）连接到 GPIO。HLC 模块根据定时事件生成中断。CLB ISR 内还会翻转一个 GPIO。通过间接访问 CLB 寄存器，可以更新定时器的事件匹配值和活动计数器寄存器，从而改变定时器频率。

4.2.8 示例 10 – 双状态定时器

本示例的定时器配置与前一示例相同。不同之处在于使用 FSM 子模块翻转 CLB 输出，并将其连接到 GPIO。FSM 模块充当单比特存储块。中断的配置方式与前一示例相同。比较 CLB 输出与 ISR 中翻转的 GPIO，可观察 CLB 中断延迟。

4.2.9 示例 11 – 中断标记

本示例为定时器设置两个不同的匹配值。HLC 子模块利用这两个事件生成中断。中断 TAG 用于区分由 CLB 计数器 match1 事件和 match2 事件生成的中断。

4.2.10 示例 12 – Output Intersect（输出交集）

本示例对 CLB 模块的配置与 external_AND_gate 示例相同。不过，输出不再通过 Output X-BAR 导出到 GPIO，而是替换 ePWM1 的输出，以此将信号送至 GPIO。具体做法是将 GPIO 配置为 EPWM1A 输出，然后启用输出交集功能。

Copyright © 2023 Texas Instruments Incorporated

4.2.11 示例 17 – 单次触发 PWM 生成

本示例演示如何将 CLB 逻辑单元配置为单次触发 PWM 生成器。示例使用组合逻辑（LUT）、状态机（FSM）、计数器和 HLC，展示收到外部或软件触发后，CLB 如何生成单次触发 PWM 输出。



图 4-13. 示例 17：单次触发 PWM 输出

CLB 逻辑单元配置为单次触发定时器：收到触发信号后，定时器从零开始计数，达到 MATCH 值后停止，直至收到下一次触发。计数期间输出为 HIGH；计数器达到 MAX 并停止后，输出变为 LOW。上述逻辑由 LUT、FSM 和计数器实现。另一个计数器用于确保后续系统仅响应上升沿事件，而不是输入电平。此示例还通过 HLC 子模块和 CLB 中断机制支持可变脉冲宽度。HLC 每在另一计数器记录到 3rd 个触发事件后生成一次中断，脉冲宽度则由应用程序更新。示例配置的输出脉冲宽度范围为 0.2 µs 至 0.8 µs，每次中断 ISR 后递增 50 ns。PWM 寄存器配置为使用活动寄存器和影子寄存器，这同样由 HLC 块完成。

CLB 整体配置如图 4-14 所示。

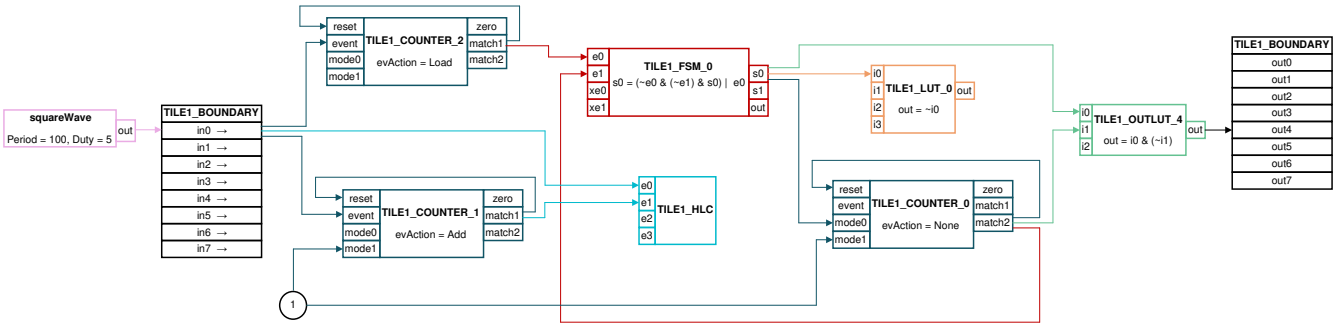


图 4-14. 示例 17：CLB 总体配置

示例支持两种配置模式：软件触发和外部信号触发。可将 EXAMPLE_MODE 定义设为 0/1，以选择所需模式。软件触发模式下，可在 CCS Expression（表达式）窗口中手动将 SOFT_TRIGGER 从 0 改为 1，并通过示波器观察单次脉冲输出。请确保先将变量设为“0”再改为“1”，因为 CLB 系统仅响应上升沿。外部信号触发模式下，EPWM 模块配置为生成 1 MHz 触发信号，其导通时间很短（占空比为 10%）。该 EPWM 信号从 GPIO0 输出后，在内部路由为 CLB 的触发输入，因此无需外部连线。

按照以下步骤运行此示例：

1. 在 CCS v9.0 或更高版本中，单击“Project -> Import CCS Projects…”。2. 转到 CLB 工具示例目录。路径为：a. [C2000Ware]\driverlib\f28004x\examples\clb\ccs。以下说明均假定使用上述 C2000Ware 目录。3. 选择“clb_ex17_one_shot_pwm”项目，然后单击“Finish”（完成）。4. 在 CCS Project Explorer（项目资源管理器）窗口中展开“clb_ex17_one_shot_pwm”项目，并打开“clb_ex17_one_shot_pwm.syscfg”文件。

Copyright © 2023 Texas Instruments Incorporated

5. 检查 tile 配置，并查看 LUT_0 和 FSM_0 中的逻辑表达式、COUNTER_0、COUNTER_1、COUNTER_2、HLC 以及输出 LUT 的配置。6. 将 EXAMPLE_MODE 配置为 0/1，以选择软件或外部触发模式。7. 在 CCS 菜单中选择“Project -> Build Project”（构建项目）。8. [Optional] 有关如何运行 CLB 仿真的说明，请参阅第 3.5.3 节。9. 将示例加载到 F28004x 控制卡上。

10. 如果选择软件触发模式：
    a. 将 SOFT_TRIGGER 变量添加到 CCS Expression（表达式）窗口。b. 将 GPIO2 连接到示波器，并确保示波器处于“One-shot”模式，而不是“FREE_RUN”模式。c. 运行示例，并在窗口中设置 SOFT_TRIGGER = 1；此时应能通过示波器观察到单次脉冲。d. 先将 SOFT_TRIGGER = 设为 0，再将 SOFT_TRIGGER = 设为 1，以生成下一个脉冲。e. 每次都重复上述步骤，以提供上升沿触发。f. 每触发三次，脉冲宽度会增加 50 ns。

11. 如果选择外部触发模式：
    a. 将 GPIO0（触发信号）和 GPIO2（输出）连接到示波器，并将示波器设为 FREE_RUN 模式。每逢 3rd 个触发信号上升沿后，应能观察到输出脉冲宽度呈线性变化。

4.2.12 示例 21 – 时钟分频器与 NMI

本示例使用 CLB 模块的时钟分频器对 CLB 时钟进行分频，并将分频后的时钟作为输入送入 tile 逻辑。示例还使用 HLC 模块生成 NMI 中断。该模块仅适用于 CLB Type 2 及更高类型。

4.2.13 示例 22 – 串行器

本示例将 CLB COUNTER 配置为串行器模式，使其充当移位寄存器。该模块仅适用于 CLB Type 2 及更高类型。

4.2.14 示例 23 – LFSR

本示例将 CLB COUNTER 模块配置为线性反馈移位寄存器（LFSR）模式。该模块仅适用于 CLB Type 2 及更高类型。

4.2.15 示例 24 – 锁定输出掩码

本示例使用 CLB 的输出锁定掩码功能，锁定所选输出信号的覆盖设置。该模块仅适用于 CLB Type 3 及更高类型。

4.2.16 示例 25 – 输入流水线模式

本例启用了 CLB 输入流水线（Input Pipeline）模式，可将输入信号延迟一个时钟周期。该模式仅适用于 CLB Type 3 及更高类型。

4.2.17 示例 26 – 时钟流水线模式

本例启用了 CLB 流水线模式，该模式会影响 CLB COUNTER 和 HLC 的行为。该模块仅适用于 CLB Type 3 及更高类型。

4.3 专家示例

这些示例旨在深入展示 CLB 的应用场景。每个示例都会说明如何配置子模块以实现特定功能。第 2.1 节列出的链接中还提供了更多示例。

若要更好地理解 Tile Design 的配置及子模块之间的连接关系，请生成 一幅 CLB 方框图；具体步骤见第 3.4 节。

4.3.1 示例 27 – SPI 数据导出

本示例使用 CLB 的高速数据导出功能，并将一个 HLC 寄存器从 CLB 模块通过 SPI RX 缓冲区导出。该模块仅适用于 CLB Type 3 及更高类型。

Copyright © 2023 Texas Instruments Incorporated

4.3.2 示例 28 – SPI 数据导出 DMA

本示例使用 CLB 的高速数据导出功能，通过 SPI RX 缓冲区从 CLB 模块导出一个 HLC 寄存器。SPI RX 缓冲区接收的数据通过 DMA 传输到存储器。该模块仅适用于 CLB Type 3 及更高类型。

4.3.3 示例 29 – 时间戳

本示例演示如何为 CLB 生成的中断添加时间戳。ePWM1 触发时会生成中断。ePWM1 配置为由 TZ1 和 TZ2 触发中断，这两个信号都是单次触发跳闸源。

4.3.4 示例 30 – 循环冗余校验

本示例演示如何将 CLB 逻辑单元配置为循环冗余校验（CRC）模块。该模块模拟 CRC 过程，以检测原始数据因意外变化而产生的错误。该模块
仅适用于 CLB Type 2 及更高类型。

注意：CRC 实现的位数最多为 31 位。

4.3.5 CLB TDM 串行端口

文件：clb_ex31_tdm_serial_port.c

有关此示例的详细说明，请参阅应用笔记《使用 CLB 实现自定义串行接口》（SPRAD62）。

本示例使用一个 CLB 逻辑单元（tile）接收 TDM 数据流并生成 TDM 输出流。接收到四个 32 位字后，CLB 会向 CPU 生成中断。CPU 可将四个 32 位值加载到 CLB
FIFO 中以供传输。CLB 和 CPU 均配置为以最高速度运行。

本示例仅适用于 CLB Type 2 及更高类型的 C2000 MCU 器件。

External Connections TDM 输入信号 GPIO 引脚 FSYNC_IN GPIO00 BCLK_IN GPIO01
DATA1_IN GPIO02

TDM 输出信号 GPIO 引脚 FSYNC_OUT GPIO04 BCLK_OUT GPIO05 DATA1_OUT GPIO06

4.3.6 CLB LED 驱动器

文件：clb_ex32_led_driver.c

有关此示例的详细说明，请参阅应用笔记《使用 CLB 实现自定义串行接口》（SPRAD62）。

本示例使用两个 CLB 逻辑单元（tile）与 LP5891-Q1 LED 驱动器通信。一条 CCSI 总线用于按照 CCSI 总线协议发送数据，另一个逻辑单元用于接收 CCSI 总线数据。C28x CPU 通过硬件抽象层（HAL）与 CLB 逻辑通信。此外，本示例还
使用 PWM 生成所需的 CCSI 时钟，并使用定时器向 LED 驱动器生成周期性同步事件。

本示例仅适用于 CLB Type 2 及更高类型的 C2000 MCU 器件。

External Connections CCSI 输入信号 GPIO 引脚 LED 驱动器 CLB_SI
N_1 GPIO17 SOUT

CCSI 输出信号 GPIO 引脚 LED 驱动器 CLB_SOUT_1 GPIO08 SIN CLB_SCLK_1 GPIO01 SCLK

4.3.7 FPGA/CPLD 迁移到 C2000 的示例

有关如何将基于 FPGA 或 CPLD 的自定义逻辑迁移到 C2000 器件的 CLB 外设，请参阅
How to Migrate Custom Logic From an FPGA/CPLD to C2000 Microcontrollers Application Report 文档中的更多示例。

Copyright © 2023 Texas Instruments Incorporated

5 在现有 DriverLib 项目中启用 CLB 工具

按照以下步骤将 CLB 支持添加到现有的 C2000WARE DriverLib 项目：

1. 从 CLB 示例文件夹中复制“empty.syscfg”文件并放入项目目录（F2837xD 对应路径为 <C2000WARE_INSTALL>\driverlib\f2837xd\examples\cpu1\clb\empty.syscfg）。2. CCS 会询问是否启用 SysConfig。接受提示并选择“Yes”（是）。

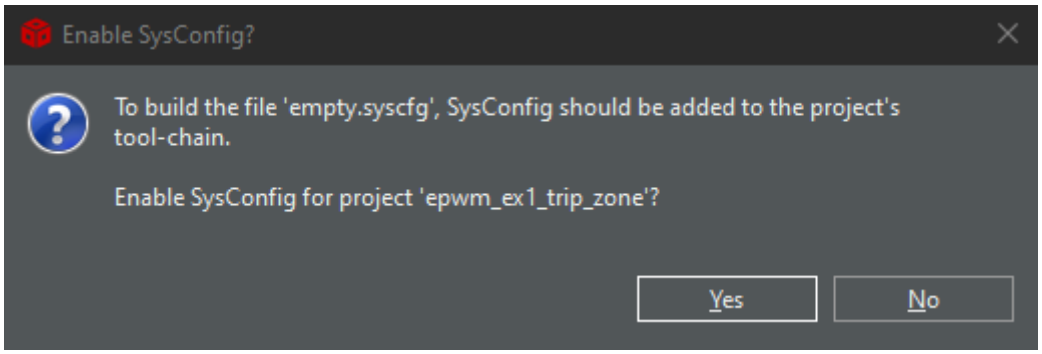


图 5-1. 启用 SysConfig

3. 打开“Project Properties”（项目属性），然后依次选择“Resources”→“Linked Resources”（链接资源）。添加以下变量路径：

a. C2000WARE_ROOT [PATH_TO_C2000WARE] b. CLB_SYSCFG_ROOT [PATH_TO_CLB_TOOL] i. 详细信息请参阅步骤 6。

4. 在“Project Properties”（项目属性）窗口中，选择“Build”→“Steps”。5. 按照图 5-2 所示，将以下命令添加到构建后步骤：a. ${NODE_TOOL} "${C2000WARE_ROOT}/dot_file_libraries/clbDotUtility.js" "${C2000WARE_ROOT}" "$ {BuildDirectory}/syscfg" "${BuildDirectory}/syscfg/clb.dot"



图 5-2. 构建后步骤

Copyright © 2023 Texas Instruments Incorporated

6. 接下来，打开“Resources” → “Linked Resources”，确认 CLB_SYSCFG_ROOT 路径正确。然后依次选择“Build” → “SysConfig” → “Basic Options”，并在 Root system config metadata 列表中添加正确路径；仅当项目尚未包含 SysConfig 时才执行此操作：

a. 确认 Linked Resources（链接资源）中的 CLB_SYSCFG_ROOT 路径正确。

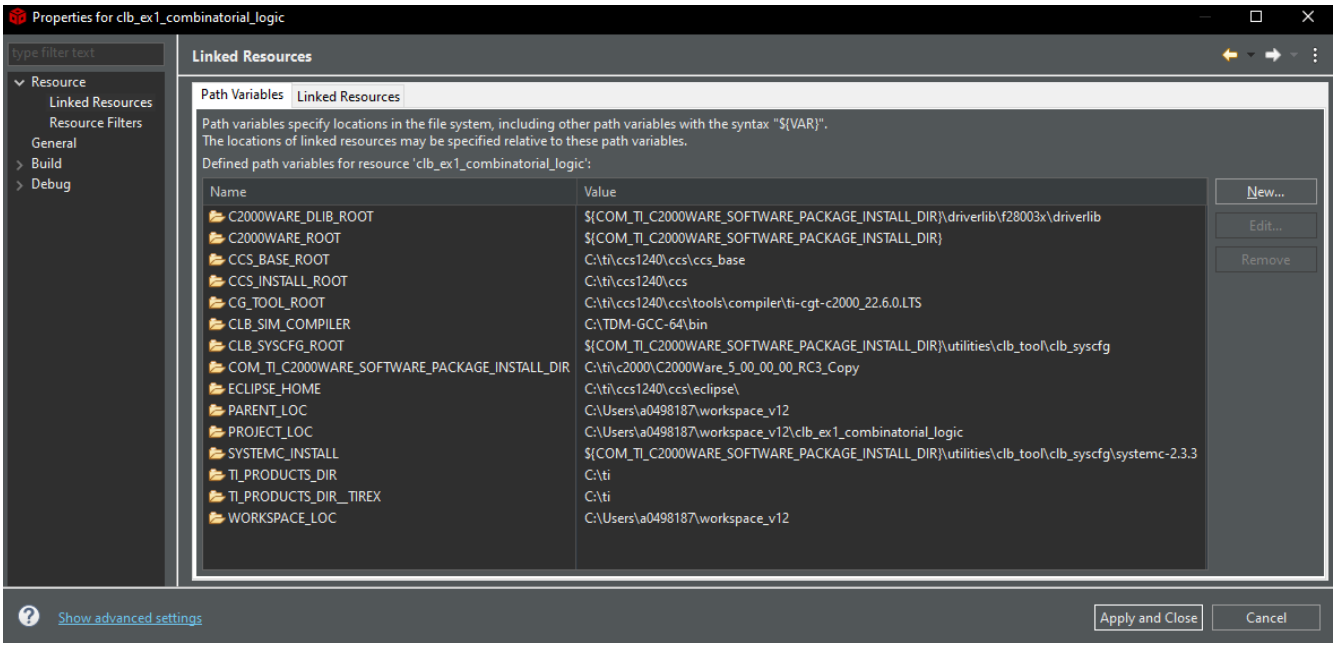


图 5-3. 启用 CLB 工具所需的链接资源

b. ${CLB_SYSCFG_ROOT}/.metadata/product.json

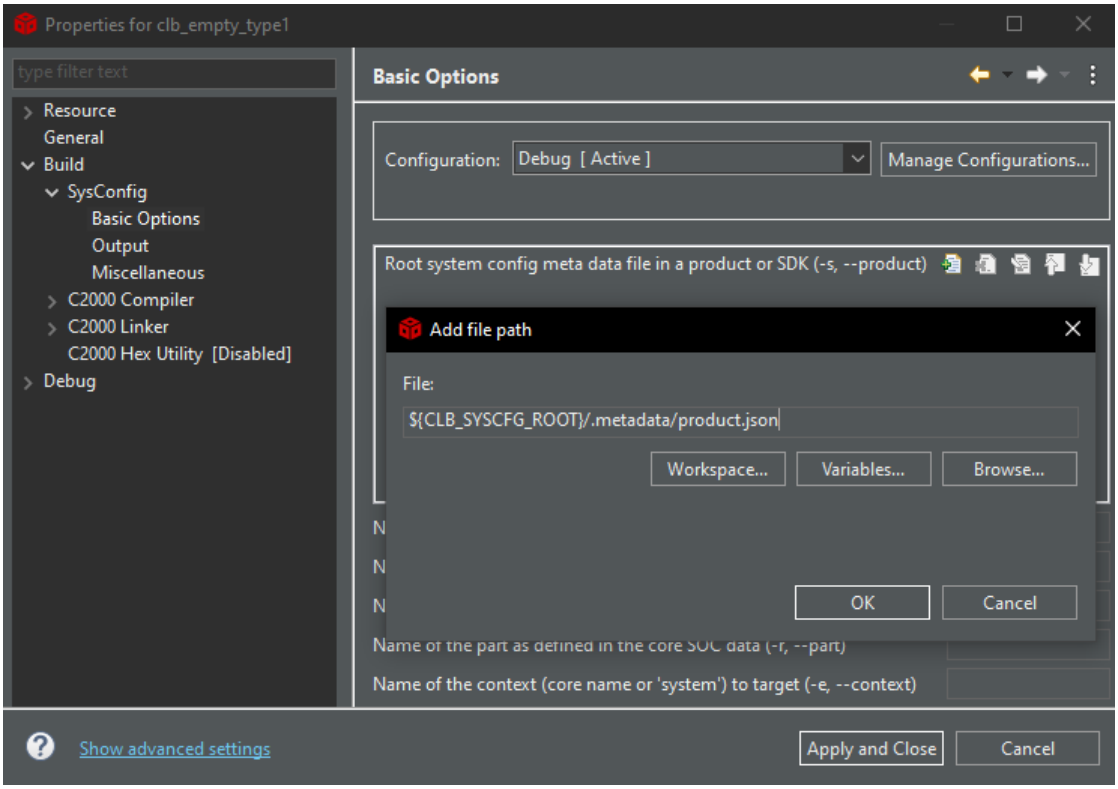


图 5-4. SysConfig SDK 路径

Copyright © 2023 Texas Instruments Incorporated

7. 最后单击“Apply”（应用）和“Close”（关闭）。

Copyright © 2023 Texas Instruments Incorporated

8. 构建项目后，CLB 工具生成的内容会出现在构建配置目录中。图 5-5 展示了在 epwm_ex1_trip_zone DriverLib 示例中添加 CLB 支持后的情况。

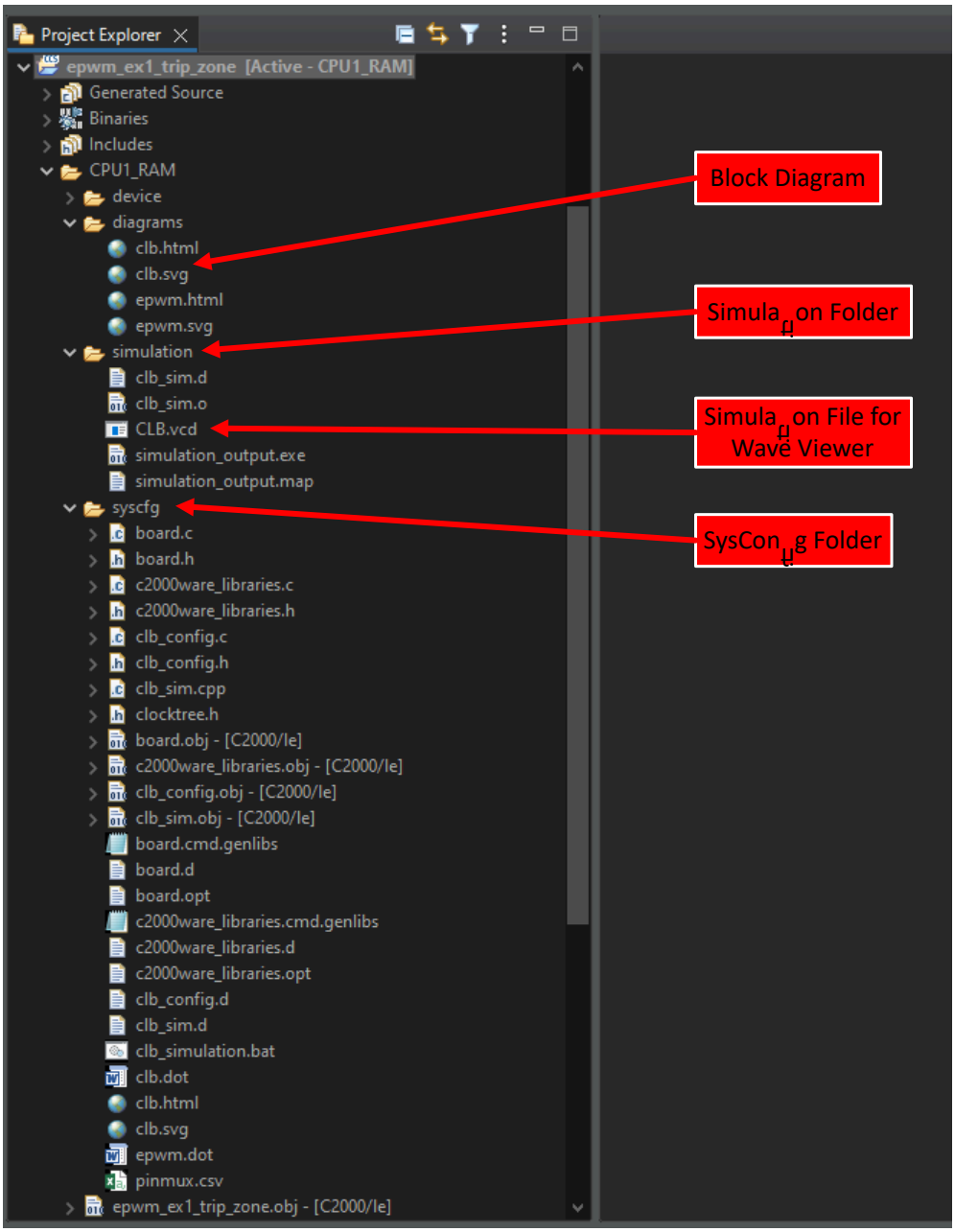


图 5-5. 支持 CLB 工具的 epwm_ex1_trip_zone

Copyright © 2023 Texas Instruments Incorporated

6 常见问题解答（FAQs）

问题：现有 CLB 项目与最新 CLB 软件包不兼容。CCS Problems（问题）窗口中出现以下构建错误：“No such resource: /TILE.syscfg.js”。

回答：SysConfig 最近的更改要求修改并更新项目的 .syscfg 文件，以反映 CLB 软件包中 TILE 资源的新位置。

注意

若要更新该文件，需要使用文本编辑器修改 .syscfg 文件。

将以下代码行：

var TILE = scripting.addModule("/TILE");

替换为

var TILE = scripting.addModule("/utilities/clb_tool/clb_syscfg/source/TILE");

问题：在我的计算机上使用 CLB 工具需要满足哪些要求?

回答：最低要求如下：

- C2000Ware 2.00.00.03
- CCSv9.1.0.00010
- Windows 64-bit、Linux 或 Mac

问题：生成 CLB 仿真 .vcd 文件时，控制台会显示许多警告，需要处理吗?

回答：这些警告可以忽略。它们由编译指令中使用的调试消息标志产生，不会影响功能。只需处理错误。

```
../syscfg/clb_sim.cpp: In constructor 'Top::Top(const sc_core::sc_module_name&, sc_core::sc_trace_file*)':
../syscfg/clb_sim.cpp:462:25: warning: 'Top::TILE1_AOC_7_release_customWave' will be initialized after [-Wreorder]
    customWave          TILE1_AOC_7_release_customWave;
                        ^
../syscfg/clb_sim.cpp:554:50: warning:   base 'sc_core::sc_module' [-Wreorder]
        sc_top_clock("sc_top_clock", 20000, SC_PS)
                                             ^
../syscfg/clb_sim.cpp:469:5: warning:   when initialized here [-Wreorder]
    Top(const sc_module_name &name, sc_trace_file *_tf):
    ^
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
        sc_top_clock("sc_top_clock", 20000, SC_PS)
                                             ^
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
../syscfg/clb_sim.cpp:554:50: warning: deprecated conversion from string constant to 'char*' [-Wwrite-strings]
```

图 6-1. 仿真警告示例

Copyright © 2023 Texas Instruments Incorporated

7 修订历史

注意：早期修订版中的页码可能与当前版本不同。

从修订版 A（2020 年四月）到修订版 B（2023 年七月）的更改	页
• 更新了全文中的表格、插图和交叉引用编号格式	3
• Changes were made through the entire User's Guide due to major updates to the CLB Tool	3

Copyright © 2023 Texas Instruments Incorporated

重要通知与免责声明

TI 提供的技术和可靠性数据（包括数据表）、设计资源（包括参考设计）、应用或其他设计建议、网络工具、安全信息及其他资源均“按原样”并连同所有缺陷提供。TI 不作任何明示或默示保证，包括但不限于适销性、适用于特定用途或不侵犯第三方知识产权的默示保证。

这些资源面向使用 TI 产品进行设计的专业开发人员。您应独自负责以下事项：(1) 为您的应用选择合适的 TI 产品；(2) 设计、验证并测试您的应用；以及 (3) 确保您的应用符合适用标准及其他安全、安保、法规或其他要求。

这些资源可能会在不另行通知的情况下变更。TI 仅允许您将这些资源用于开发使用资源中所述 TI 产品的应用。禁止以其他方式复制或展示这些资源。TI 不授予您任何其他 TI 知识产权或第三方知识产权的许可。对于因您使用这些资源而产生的任何索赔、损害、费用、损失和责任，TI 不承担责任；您应向 TI 及其代表提供充分赔偿，使其免受上述事项影响。

TI 产品受 TI 销售条款或其他适用条款约束；相关条款可在 ti.com 上查阅，也可能随 TI 产品一并提供。TI 提供这些资源不会扩大或以其他方式改变 TI 产品适用的保证或免责声明。

TI 反对并拒绝您可能提出的任何附加条款或不同条款。 重要通知

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2023, Texas Instruments Incorporated